

Spark

Bigdata5 : Spark

- การทำ batch process ของ MapReduce ไม่ได้เหมาะกับ workflow ในบางงาน เช่น iterative (การทำงานวนซ้ำ) หรือ interactive computing (ตอบสนองในเวลาที่กำหนด)
- เพราะ MapReduce จะใช้การ pull execution คือ ระหว่างการทำงานในแต่ละ step ของ MapReduce Task จะมีการ read/write ไปที่ HDFS เสมอ ทำให้การทำงานแต่ละ step จะต้องใช้เวลา ไม่สามารถทำ step ต่อไปได้ทันที (too slow for interactive)
Map >>> write >>> HDFS >>> read >>> sort&shuffle >>> write >>> HDFS >>> read >>> reduce
- Spark ก็เข้ามาตอบโจทย์ตรงนี้ เพราะ spark เป็น fast, general purpose distributed computing ที่ไม่ต้องแตก job ใน data work ให้เป็น MapReduce Job ต่างๆ
- โดย spark จะสามารถทำงาน high speed ได้ด้วยการใช้ data model แบบใหม่ที่เรียกว่า RDD หรือ resilient distributed datasets ซึ่งจะเก็บใน memory ทำให้ทำงานเร็วกว่าการทำงานผ่าน disk 10 – 100 เท่า
- ข้อเสียคือทำให้ server ต้องมี memory ที่มากขึ้นและข้อมูลใน memory จะไม่ permanent ต้องมีกระบวนการ recovery เมื่อเกิดการ fail ของ server

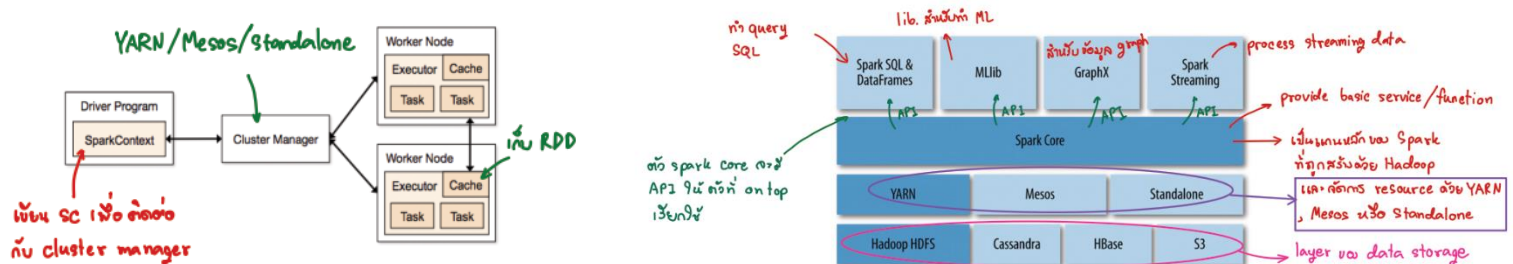
Spark Basic

- Spark จะมี API สำหรับการทำ distributed programming คล้ายๆกับ MapReduce แต่จะออกแบบมาให้ทำงานได้เร็วเพื่อการทำ interactive query และ iterative algorithm ซึ่งจะเป็นส่วนสำคัญสำหรับการทำ optimization และ machine learning
- ซึ่งจะเก็บ RDD ที่ต้องการใช้ process ไว้ใน memory ซึ่งตัว RDD จะถูกเก็บได้ทั้งใน memory และ disk ส่วนที่จะ process >>> load เข้า memory / ส่วนที่ยังไม่ใช้ >>> เก็บไว้ใน disk ก่อน (roll-in-roll-out)
- นอกเหนือจากการ support MapReduce operation แล้ว ยังมี API สำหรับ process อื่นๆ ด้วยเช่น sample, filter, join, collect
- Spark ถูกพัฒนาด้วยภาษา Scala แต่มี API ที่ใช้กับพวก Python, R, Java ได้
- Spark จะเก็บ data ที่จะใช้ไว้ใน memory ให้มากที่สุด เพื่อป้องกันการต้อง reload เข้า/ออก memory หลายครั้ง ในระหว่างการทำ iteration process
- ตัว spark program จะมีลักษณะเป็น data flow graph เรียกว่า Directed Acyclic Graph (DAG) ซึ่งจะเป็นหลายๆ node เชื่อมกันและมีทิศทางในการทำงาน node สู่ node ไม่มีการวนซ้ำระหว่าง node (ไม่มี loop ระหว่าง node)
- เมื่อจะ execute ตัว spark จะรู้ว่า DAG เป็นยังไง/ ทำอะไร/ ที่ node ไหน/ ใช้ resource เท่าไหร่ เพื่อ manage การกระจาย data และ resource ในแต่ละ node รวมถึง schedule การทำงานได้
- Spark จะ focus ที่การ computation มากกว่าการ storage data
- Spark สามารถ run ใน cluster ที่เป็น data warehouse และ cluster management เช่น

1.) Hadoop with HDFS และ YARN

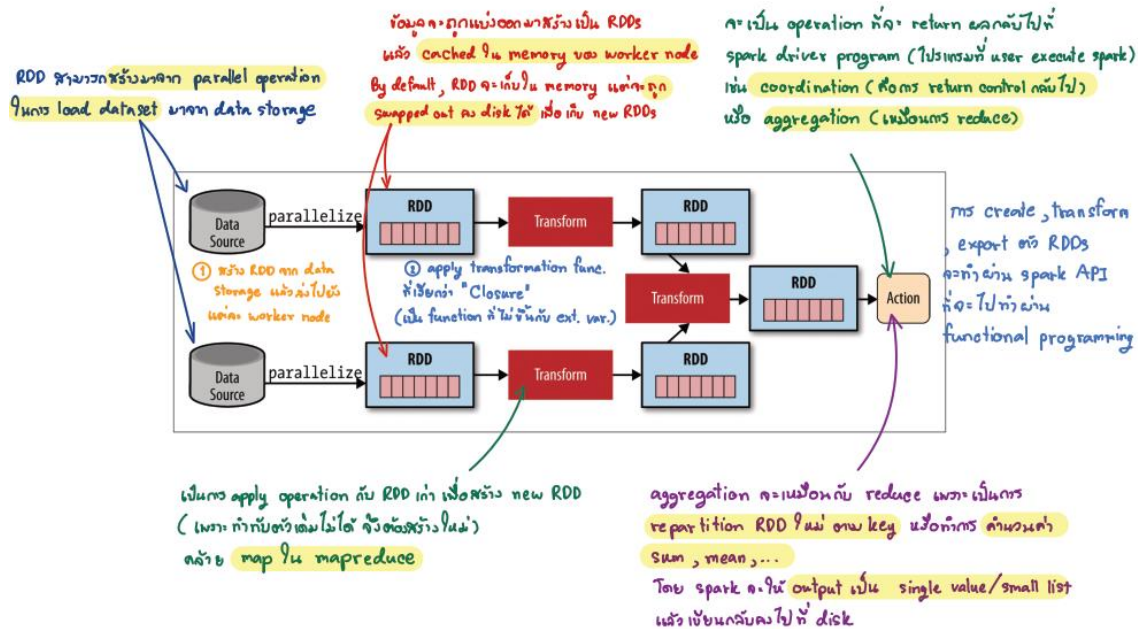
2.) Mesos (Apache Cluster Management Software) สามารถ run ได้ถึง 10,000 node เพื่อ support application เช่น Hadoop, Spark, Kafka, Elastic search ด้วย API สำหรับการทำ resource management และ scheduling

3.) Standalone เป็นแบบง่ายสุด มีแค่ 1 master กระจายงานไปหลาย worker node (ไม่ต้องไปพึ่ง YARN หรือ Mesos)



Resilient Distributed Dataset (RDD)

- RDD คือ collection ของ object แบบ read – only (immutable / write once read many) ที่จะแบ่งแต่ละ set ไปตาม set ของ machine (node) เช่น มี 1000 object ก็แยกไป 10 node >> node ละ 100 object ป้องกันปัญหาตอนอยาก update พร้อมๆกัน *** ข้อจำกัด >>> จะเหมือนกับ HDFS คือไม่สามารถ update แบบ in-place ได้ ต้องสร้าง object ใหม่

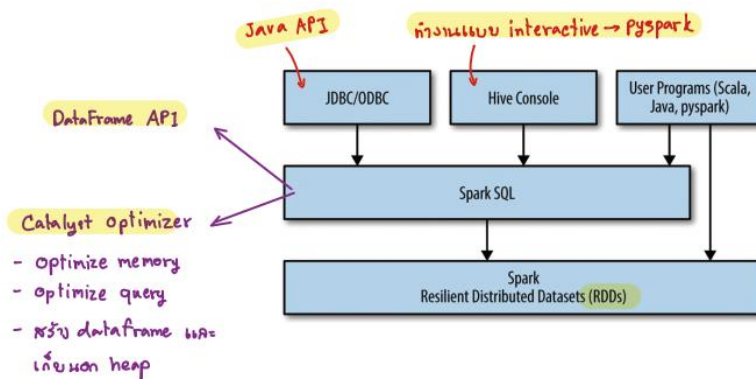


- การทำงานของ spark จะเรียกว่า **lazy-execution** คือการที่ตัว spark จะดูว่า sequence ของการทำ transformation และ action มีอะไรบ้าง แล้วบันทึกไว้ จะทำงานเมื่อมีการ submit job เข้ามาที่ cluster >> transformation จะบันทึกไว้ว่าทำอะไรบ้าง พอ action เข้ามาถึงเริ่มทำ
- **Lazy-execution** จะเป็นการทำเพื่อ optimization การคำนวณ เมื่อรู้ sequence งานทั้งหมดก่อน execute ก็จะทำให้รู้ว่า การทำงานในส่วนไหนต้องทำอะไร ใช้ data อะไรบ้าง และใช้ resource เท่าไหร่
- **Closure** จะเรียกใช้ local variable ในตัว ไม่ขึ้นกับ global variable แต่ถ้าต้องการใช้ global variable เพื่อ share data ระหว่าง closure ใน worker node เดียวกัน หรือต่างกัน จะสามารถทำได้ผ่าน share variable ได้ 2 ประเภท
 - 1.) **Broadcast variable** >> จะเป็นการ share ระหว่าง server ซึ่งการ update พร้อมๆกันหลายที่จะมีปัญหา จึงจะจำกัดว่าเป็น read-only ส่วนใหญ่จะมาเป็นรูปของ lookup table หรือ shared lists ให้แต่ละ closure เห็นและดึงไปใช้งาน
 - 2.) **Accumulators** >> เป็นตัวแปรที่แต่ละ worker node จะสามารถ "add" ค่าได้ เช่น $x += 1$ ซึ่งใช้เป็นตัว counter ซึ่งเป็น associate operation ดังนั้นจะไม่สำคัญว่าใครบวกก่อน/หลัง ค่าสุดท้ายยังงี้ก็เท่ากัน
- Spark ก็จะมี support เรื่อง fault-tolerant เหมือนกับ HDFS โดย
 - 1.) แต่ละ RDD ก็จะมีบันทึก operation และ flow การทำงานต่างๆ เรียกว่า **linage graph** ซึ่งเกิดจาก lazy operation
 - 2.) เมื่อ node ไหนเกิด fail และ partition ใน RDD หาย ก็จะสามารถ re-compute ขึ้นใหม่ได้ จากการใช้ linage graph
 - 3.) Save ข้อมูลลง disk ด้วยคำสั่ง persistent() เพื่อสร้าง copy ใน disk เมื่อเกิดปัญหาที่จะดึงข้อมูลใน disk มาใช้ได้
- การทำงานของ spark จะเร็วพอที่จะทำงานแบบ interactive (ทำแล้วแทบจะเสร็จทันที) ด้วย Python REPL (Read-Evaluate-Print-Loop) เรียกว่า PySpark

Bigdata6 : Spark SQL

Spark Data Frame

- เป็น distributed collection ที่เก็บข้อมูลในรูป row และ column (คุณสมบัติเหมือน RDD) เหมือนกับ table ใน database หรือ excel sheet โดยมีคุณสมบัติ 3 ข้อ
- 1.) **Immutable** >> DataFrame/ RDD ไม่สามารถถูกแก้ไขได้ แต่สามารถถูก transform เป็นตัวใหม่ได้ เมื่อ apply transform operation
- 2.) **Lazy Evaluate** >> Task จะยังไม่ทำงานจนกว่าเราจะสั่ง action operation และ re-play เมื่อเกิด fail ด้วย lineage graph ได้เหมือน RDD
- 3.) **Distributed among data node** >> สามารถทำงานแบบกระจายไปยังหลาย worker node ได้
- Spark DataFrame ถูกออกแบบมาสำหรับประมวลผล collection ขนาดใหญ่ ทั้งในรูปของ structured หรือ semi-structured data
- โดยจะถูกเก็บใน **memory** เหมือนกับ RDD แต่ต่างกันว่า DataFrame จะมี custom memory management เพื่อให้ทำงานได้เร็วขึ้น
- DataFrame จะเก็บ **data นอก heap** (Java Heap Space คือพื้นที่เก็บ object/ code ของโปรแกรมที่ถูกสร้างขึ้นโดย java จะมีการทำ garbage collection เพื่อเอาข้อมูลส่วนที่ไม่ใช้ใน heap ออก เพื่อ compact เนื้อที่ เพื่อเพิ่มพื้นที่ว่าง ซึ่ง garbage collection จะใช้เวลาในการทำ) เมื่อ DataFrame เก็บนอก heap ก็จะต้องลดงานส่วนที่จะต้องทำ **garbage collection** ทำให้เก็บ data ใน memory ได้มากขึ้น >> ทำงานได้เร็วขึ้น <https://saixiii.com/what-is-java-heap-memory/>
- DataFrame จัดเก็บในรูปแบบ **column** จึงช่วยให้ spark รู้ **schema** /แต่ละ column เป็น type อะไร ทำให้ **optimize** และ **re-arrange execution plan** เพื่อให้กิน **resource** น้อยลง/ ทำงานได้ไวขึ้น (เก็บ data เฉพาะที่ใช้ ไม่ต้อง load มาทั้งหมด >> เก็บใน memory ได้หมด ไม่ต้อง load จาก disk)
- DataFrame จะ access ผ่าน spark sql ซึ่งมี API support หลายภาษา เช่น python, R, Scala, Java



Bigdata7 : Machine Learning

Machine Learning

- Machine Learning คือ Task ที่เป็นการอนุมาน pattern และ relationship ระหว่าง ตัวแปร และ ข้อมูลใน dataset โดย output ของ machine learning จะเป็น model ที่สามารถจับ pattern และ relationship ของชุดข้อมูลออกมาได้ แบ่งออกเป็น

Supervised machine learning >> เป็น model ที่บอก relationship ระหว่าง **dependent(x1,x2,x3,...)** และ **independent variable(y)** เพื่อนำมา **predict** ค่าของ **dependent variable**

- Feature หรือ variable สามารถเป็น Numeric data >>> predict by regression / Category data >>> predict by classification
- ในการทำ supervised learning จะแบ่ง dataset ออกเป็น 2 ส่วน คือ 1.) Training data ใช้สำหรับ train model เพื่อไป predict class label หรือ dependent variable 2.) Testing data ใช้สำหรับ validate หรือ evaluate ประสิทธิภาพในการ predict ของ model

Unsupervised machine learning >> เป็น model ที่จะจับ hidden pattern หรือ structures ของ dataset โดยที่เราไม่มีการ label ข้อมูลเอาไว้ล่วงหน้า เช่น clustering, association rules

Machine Learning Applications

- **Classification** >> เป็นการสร้าง model เพื่อ predict class label
- **Regression** >> เป็นการสร้าง model เพื่อ predict numerical label หรือ dependent variable
- **Clustering** >> เป็นการแยก dataset ออกเป็น clusters หรือ segment โดยใช้ความคล้ายคลึงกันของข้อมูล ว่ามีการรวมกลุ่มหรือกระจายกันอย่างไร
- **Anomaly Detection** >> เป็นการ detect data ที่มีความผิดปกติ หรือ outliers (unsupervised) เช่น Fraud detection
- **Recommendation** >> แบ่งออกเป็น
 - 1.) **Collaborative filtering** คือการ predict rating จากพฤติกรรมมาให้ rating ของลูกค้าแต่ละราย แล้วนำสินค้าที่เกี่ยวข้องมาแนะนำให้
 - 2.) **Content-based** จะไม่ได้ใช้แค่ข้อมูล rating แต่จะใช้ข้อมูลที่มีความจำเพาะเจาะจง เช่นข้อมูลสมาชิก มาหาความสัมพันธ์กับสินค้า และแนะนำให้
- **Dimension Reduction** >> คือการลดจำนวน feature ที่มีความเกี่ยวข้องกับการทำนายผลต่ำ ทำให้เราได้ dataset ที่ dimension ลดลง แต่ยังคงรักษา character ของ original ได้ จะมีประโยชน์ในการลดปัญหา overfitting / ลดเวลาในการ train

Machine Learning with Spark

- การทำ ML ใน spark จะมีอยู่ 2 libraries
 - 1.) **MLlib** >> มี function ให้ทำเยอะกว่า spark ML
 - 2.) **Spark ML** (Pipeline API)
- ทั้งสอง lib สามารถ scalable ได้ สามารถใช้ได้กับ bigdata
- MLlib จะเก็บข้อมูลในรูปของ RDD ส่วน Spark ML จะเก็บข้อมูลในรูปของ DataFrame
- โดยจะ provide high-level API สำหรับการทำให้ ML และ statistical analysis เช่น summary statistical analysis, correlations, stratified Sampling, hypothesis testing, random data generation, kernel density estimation จะสามารถใช้ได้กับภาษา Scala, Java, Python และ R
- MLlib สามารถใช้ได้กับ machine learning task ทั่วไป ได้แก่
 - 1.) **Regression and Classification** >>> Linear Regression, Logistics Regression, Support Vector Machine, Naïve Bayes, decision Trees, Random Forest, Gradient-boosted trees, Isotonic Regression
 - 2.) **Feature Extraction and Transformation** >>> เป็นการ pre-process ข้อมูลให้อยู่ในฟอร์มที่เราจะนำไปใช้งานได้ (feature extraction เป็น dimension reduction ประเภทหนึ่ง) เช่น TF-IDF แปลง text ให้อยู่ในรูป frequency , Word2Vec แปลง text ให้เป็น vector, Standard Scaler หรือ Z-Score ทำ standardization, Normalizer ทำ normalization, Chi-Squared Feature Selection เป็น feature extraction based on ค่า chi-squared, Elementwise Product
 - 3.) **Frequent Pattern Mining** >>> หาว่าสินค้าหรือ pattern ใหนที่มักเกิดขึ้นพร้อมๆกัน เช่น FP-growth, Association Rules, Prefix Span (Sequential Pattern Mining)
 - 4.) **Recommendation** >>> Collaborative Filtering using ALS
 - 5.) **Clustering** >>> K-mean

- MLlib จะมี data abstraction(data type ชนิดพิเศษ) เริ่มต้นอยู่ 3 ประเภท ได้แก่ 1.) Vector 2.) Labeled Point 3.) Rating

1.) **Vector Type** จะมีลักษณะเป็น array ที่จะเก็บ data type ชนิด Double-Type (ทศนิยม 2 ตำแหน่ง) โดยสามารถที่จะระบุตำแหน่งของข้อมูลได้โดยใช้ index ที่เป็นชนิด integer โดยเริ่มจากค่า 0 ตัว โดยสามารถ represent ได้ใน n-dimensional space มักใช้กับ unsupervised data

MLlib จะ support vector 2 ประเภท ได้แก่ dense และ sparse

DenseVector >>> จะใช้กับ dataset ที่มีเลข 0 ไม่มาก / dense method สร้าง array ของ double-type data และ return instance ออกมาเป็น class ของ DenseVector

```
import org.apache.spark.mllib.linalg._
val denseVector = Vectors.dense(1.0,0.0,3.0)
```

package linear algebra
ทำ array มาตอนใด

SparseVector >>> จะเป็น vector ที่จะเก็บเฉพาะ non-zero data (เพื่อประหยัดเนื้อที่) จะใช้กับ dataset ที่มีเลข 0 เยอะๆ โดย instance ของ SparseVector class นั้น จะประกอบด้วย 2 arrays คือ vector หนึ่งเก็บ index ของ non-zero data และอีก vector เก็บ value ของ non-zero data

```
import org.apache.spark.mllib.linalg._
val sparseVector = Vectors.sparse(10,
  Array(3,6),Array(100.0, 200.0))
```

3/10 element
index ที่ไม่เป็น 0
value ที่ไม่เป็น 0

```
import org.apache.spark.mllib.linalg._
val sparseVector = Vectors.sparse(10,
  Seq((3,100.0), (6,200.0)))
```

index value

For PySpark,

```
denseVec = Vectors.dense([1.0, 2.0, 3.0])
```

```
sparseVec = Vectors.sparse(4, {1: 1.0, 3: 5.5})
```

2.) **LabeledPoint** จะใช้กับ dataset ที่มีการ label มาแล้ว ประกอบด้วย label (dependent variable) ที่จะจัดเก็บในรูปของ Double-type value และ features (independent) ที่จะจัดเก็บเป็น Vector type

- Label จะแบ่งออกเป็น 2 ประเภท คือ numerical และ categorical

- สำหรับ regression นั้น label จะเป็น numerical / binary classification label จะเป็น 0.0 หรือ 1.0 / multiclass classification label จะเป็น 0.0, 1.0, 2.0, ...

```
import org.apache.spark.mllib.linalg.Vectors
import
org.apache.spark.mllib.regression.LabeledPoint
```

```
val positive = LabeledPoint(1.0,
  Vectors.dense(10.0,30.0,20.0))
val negative = LabeledPoint(0.0, Vectors.sparse(3,
  Array(0,2), Array(200.0,300.0)))
```

label
feature

```
from pyspark.mllib.linalg import SparseVector
from pyspark.mllib.regression import LabeledPoint
```

```
# Create a labeled point with a positive label and a dense
feature vector.
pos = LabeledPoint(1.0, [1.0, 0.0, 3.0])
```

```
# Create a labeled point with a negative label and a sparse
feature vector.
neg = LabeledPoint(0.0, SparseVector(3, [0, 2], [1.0, 3.0]))
```

3.) **Rating Types** จะใช้กับ recommendation algorithms ซึ่งจะประกอบด้วย 3 field คือ 1.) user name (user id) (int) 2.) product name (item id)(int) 3.) rating (double type)

```
import org.apache.spark.mllib.recommendation._
val rating = Rating(100,10,3.0)
```

user id item id rating

For PySpark,

```
rating = Rating(100,10,3.0)
```

(an instance of the Rating class with user id = 100, item id = 10, and the rating = 3.0 is created)

Regression Algorithm

MLlib จะประกอบด้วยหลาย regression algorithm ในรูปของ class และ singleton object (คือใน class จะสร้างได้ object เดียว) เช่น

1.) **Linear Regression with Stochastic Gradient Descent** ใช้สำหรับการทำ regression กับ large training dataset

Regression Linear Equation

$$y = w_0 + W^T X$$

where

$$X = (x_1, x_2, \dots, x_n)$$

$$W = (w_1, w_2, \dots, w_n)$$

Vector n ส่วน

• Minimization Objective Function

$$f(W) = \frac{1}{m} \sum_{i=1}^m L(W, X_i, y_i)$$

$$L(W, X_i, y_i) = \text{Loss Function} = \frac{1}{2} (W^T X_i - y_i)^2$$

m = size of training dataset

2.) **Penalized Regression** เป็นการทำให้ regularization เพื่อป้องกันการเกิด overfitting จะมี 3 แบบ ได้แก่

- **Ridge Regression** >> เป็นการให้ L2 regularization เพื่อป้องกัน overfitting โดยการเพิ่ม sum square coefficient เพื่อปรับลด weight ของ feature บางตัวที่มีผลต่อ model น้อยให้เข้าใกล้ 0
- **Lasso with SGD** >> เป็นการให้ L1 regularization เพื่อป้องกัน overfitting โดยการเพิ่ม sum absolute ของ coefficient เข้าไปเพื่อทำให้ weight ของ feature บางตัวที่มีผลต่อ model น้อยให้ มีค่าเป็น 0
- **ElasticNet Regression** >> จะใช้ทั้ง L1 และ L2 มารวมเป็นสมการเดียวกัน แล้วปรับ weight ในการทำ L1 และ L2 แทน

L2

- Objective function with L2 regularization

$$f(W) = \frac{1}{m} \sum_{i=1}^m L(W, X_i, y_i) + \lambda R(W)$$

$$L(W, X_i, y_i) = \text{Loss Function} = \frac{1}{2} (W^T X_i - y_i)^2$$

m = Size of training dataset

λ = Regularization parameter ≥ 0

$$R(W) = \sum_{j=1}^n w_j^2 \quad n = \text{number of variables (\# of dimensions)}$$

$$f(W) = \frac{1}{m} \sum_{i=1}^m L(W, X_i, y_i) + \alpha \left(\lambda \sum_{j=1}^n |w_j| \right) + (1 - \alpha) \left(\frac{\lambda \sum_{j=1}^n w_j^2}{2} \right)$$

$$L(W, X_i, y_i) = \text{Loss Function} = \frac{1}{2} (W^T X_i - y_i)^2$$

λ = Regularization Parameter ≥ 0

α = ElasticNet Parameter where $0 \leq \alpha \leq 1$

n = number of variables (\# of dimensions)

L1

$$f(W) = \frac{1}{m} \sum_{i=1}^m L(W, X_i, y_i) + \alpha \left(\sum_{j=1}^n |w_j| \right) + (1 - \alpha) \left(\frac{1}{2} \sum_{j=1}^n w_j^2 \right)$$

$$L(W, X_i, y_i) = \text{Loss Function} = \frac{1}{2} (W^T X_i - y_i)^2$$

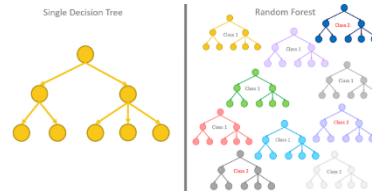
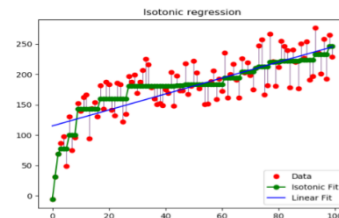
m = Size of training dataset

λ = Regularization parameter ≥ 0

ElasticNet

3.) Other Regression

- Isotonic Regression** >> จะใช้สำหรับ non-linear และเป็น non-decreasing function คือ x และ y มีการเพิ่มขึ้นเรื่อยๆ โดยจะทำการ fit เป็น step function
- Decision Tree** >> จะใช้กับงาน classification ใน data ที่เป็น label point จะใช้การตัดสินใจลักษณะแบบต้นไม้ เป็นการไล่ระดับความ impurity ของตัวแปรอิสระที่ใช้ในการตัดสินใจลงไปเรื่อยๆ ตั้งแต่ root node (impurity สูงสุด) จนถึง leaf node (impurity ต่ำสุด) เพื่อตัดสินใจว่าจะเป็น class อะไร แต่ในการทำ regression จะใช้ variance ในการวัด impurity เพื่อใช้ในการเลือก feature เพื่อพิจารณาในการตัดสินใจ โดยพยายามหาตัวแปรอิสระที่ทำให้ variance ของ y ลดลงจนเข้าใกล้ 0 จาก variance เริ่มต้นของข้อมูลทั้งหมด ค่า y ที่ class นั้นๆ จะมีทิศทางไปในทางเดียวกัน โดย leaf node อาจจะเป็นค่า mean median ของแต่ละ class
- Gradient Boosted Trees** >> เป็น ensemble method โดยใช้การ boosting โดยการสร้างต้นไม้ทีละต้น หลายๆ ต้น มาช่วยตัดสินใจ สมมติมี 100 ตัว ใช้วิธีสุ่มแล้วคืนได้ 60 ตัว เอา 60 ตัวมาสร้างต้นไม้ แล้วทำการปรับ weight ในการสุ่มไปเรื่อยๆ ถ้ารอบที่แล้วข้อมูลตัวไหนทำนายผิด ก็ให้ปรับ weight เพิ่มขึ้น เพื่อนำมาสุ่มในรอบถัดไป สุดท้ายก็จะใช้ accuracy ในการ vote ต้นไม้ที่ accuracy สูง ก็จะมี weight ในการ vote สูง
- Random Forest** >> เป็น ensemble method โดยใช้การ bagging คือ การสุ่ม data ใน training set ออกมา เพื่อให้ได้ subset ของ data มาสร้าง decision tree หลายๆ ต้น เพื่อช่วยในการตัดสินใจแล้วดู majority vote



- For a linear model such as Linear Regression with SGD, Ridge Regression, train method can be used to build a model.

- Example

```
Import org.apache.spark.mllib.linalg.Vectors
```

```
Import
```

```
org.apache.spark.mllib.regression.LabeledPoint
```

```
Import org.apache.spark.mllib.regression.LinearRegressionWithSGD
```

```
val lines = sc.textFile("data/mllib/ridge-data/lpsa.data")
```

```
val labeledPoints =
```

```
lines.map { line =>
```

```
val Array(rawLabel, rawFeatures) = line.split(',')
= line.split(',')

```

```
val features = rawFeatures.split(' ').map(_.toDouble)

```

```
LabeledPoint(rawLabel.toDouble, Vectors.dense(features))

```

```
}

```

```
labeledPoints.cache

```

```
val numIterations = 100

```

```
val lrModel = LinearRegressionWithSGD.train(labeledPoints, numIterations)

```

```
val intercept = lrModel.intercept

```

```
val weights = lrModel.weights

```

- predict** – a method that predicts a numerical label for a given set of features.

- It takes an input set of features in a form of vector and returns a numerical label of type Double.

```
val observedAndPredictedLabels =
```

```
labeledPoints.map { observation =>
```

```
val predictedLabel = lrModel.predict(observation.features)

```

```
(observation.label, predictedLabel)

```

```
}

```

```
on lrModel ก็ให้มัน predict

```

- TrainRegressor – method that trains or fits a non-linear regression model such as DecisionTree and RandomForest.

```
val categoricalFeaturesInfo = map[Int, Int]() % list of key-value pairs,
where key = id, e.g. 0, 1, 2, ..., of a variable which is a categorical
variable, value = number of categorical values for the variable, For
this example, we have an empty list means there is no categorical
variables %
```

```
val NumTrees = 3
```

```
val FeatureSubsetStrategy = "auto"
```

```
val impurity = "variance"
```

```
val maxDepth = 4 % max depth of the trees in the forest %
```

```
val maxBins = 32 % max number of intervals for discretization of real
value variables %
```

```
% ถ้า categorical Bin ๑:
ตาม class ที่ predict

```

```
val rfModel = RandomForest.trainRegressor(labeledPoints,
```

```
categoricalFeaturesInfo, NumTrees,
```

```
FeatureSubsetStrategy, impurity, MaxDepth, maxBins)
```

Classification Algorithm

1.) Linear Model เช่น

- **LogisticRegressionWithSGD** >> ปกติจะทำกับ binary class โดยการใช้ plane เพื่อมา split positive และ negative
- **LogisticRegressionWithLBFGS** (Broyden–Fletcher–Goldfarb–Shanno (BFGS) algorithm เพื่อมาหาสมการ plane ในการแก้ปัญหาแบบ nonlinear
- **SVMwithSGD**

2.) NaiveBayes

3.) Tree-based Models (non-linear) เช่น Decision Tree/ GradientBoostedTrees/ RandomForest

- วิธีการ train classification model นั้นจะใช้ RDD ที่เป็น LabeledPoint เป็น argument ของ function แล้ว return ออกมาเป็น model

```
val Array(trainingData, validationData, testData) =  
  labeledPoints.randomSplit(Array(0.6, 0.2, 0.2))  
trainingData.cache()  
val numIterations = 100  
val svmModel =  
  SVMwithSGD.train(trainingData, numIterations)  
• predict method returns a class or categorical label of  
  type Double for a given set of features in Vector format  
val predictedLabels = testData.map { observation =>  
  val predictedLabel =  
    svmModel.predict(observation.features)  
    (predictedLabel, observation.label)  
}
```

- **trainClassifier** method นั้นจะใช้สร้าง classification model ในตระกูล tree-based โดยจะใช้ RDD ที่เป็น LabelPoint และ specific hyperparameter (Numtrees, maxDepth, impurity,...) เป็น argument

การ save และ load model

The **save** method persists a trained model to a disk. It takes a **SparkContext** and **path** as arguments and saves the source model to the given path. The saved Model can be later read with the load method.

```
svmModel.save(sc, "models/svm-model")
```

The load method is defined in the companion model objects that can be imported.

```
import org.apache.spark.mllib.classification.SVMModel  
val savedSVMModel = SVMModel.load(sc, "models/svm-model")
```

Clustering

- 1.) **Kmean** >> เป็นการจับกลุ่มข้อมูลโดยใช้ระยะห่างระหว่างข้อมูล ข้อมูลที่มีระยะห่างใกล้ๆกันก็จะถูกจับเป็นกลุ่มเดียวกัน โดยมีตัวแทนของกลุ่มคือ จุด centroid ที่เป็นค่าเฉลี่ยระยะทางระหว่างสมาชิกในกลุ่มนั้นๆ

- 2.) **StreamingKmeans** >> จะใช้กับ streaming data จับกลุ่มข้อมูลจากข้อมูลเดิมที่เรามี แล้วพอ data ใหม่เข้ามาก็จะทำการจับกลุ่มใหม่เรื่อยๆ

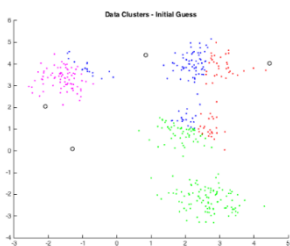
- 3.) **GuassianMixture** >> เป็นการจับกลุ่มแบบเป็น normal distribution ในพื้นที่ที่ใกล้จุด centroid จะมีความหนาแน่นของข้อมูลที่สูง โกลออกไปก็มีความหนาแน่นของข้อมูลลดลง กระจ่าย data ไป cluster อื่นๆตาม prob. แตกต่างจาก kmean ที่ใช้การปรับเข้าหา centroid เพียงอย่างเดียว

- 4.) **LDA** >> ใช้กับการ clustering document โดยการทำ topic modelling

- 5.) **Power Iteration Clustering** >> ใช้กับการ clustering ข้อมูลที่ลักษณะเป็น graph

- การ **train model** จะทำโดย clustering-related singleton object โดยการ **input RDD** ที่เป็น type **Vector** และ **specific parameter** เข้าไป (ex. RDD vector + iteration no.) แล้ว return ออกมาเป็น model

- Parameter และ type ของ model ที่ return ออกมา จะขึ้นอยู่กับว่าเป็น clustering algorithm อะไร เช่น ถ้าเป็น parameter ใน **train method** ของ **Kmean** จะประกอบด้วย **number of clusters**, **maximum number of iterations in each run**, **number of parallel runs**, **initialization modes** และ **random seed values**



K-Means Algorithm

```
import org.apache.spark.mllib.clustering.KMeans  
import org.apache.spark.mllib.linalg.Vectors  
val lines = sc.textFile("data/mllib/Kmeans_data.txt")  
val arrayOfDoubles = lines.map { line => line.split(" ").map(_toDouble) }  
val vectors = arrayOfDoubles.map { a => Vectors.dense(a) }.cache()  
val numClusters = 2  
val numIterations = 20  
val KmeansModel =  
  KMeans.train(vectors, numClusters, numIterations)
```

- Run method can also be used for clustering-related objects.

```
val numClusters = 2  
val numIterations = 20  
val kmeans = new  
  KMeans().setMaxIterations(numIterations).setK(numClusters)  
val KMeansModel = kmeans.run(vectors)
```

- **Predict** method returns a cluster index for a given observation. It takes a Vector as an argument and return a value of type Int.

```
val obs = Vectors.dense(0.0, 0.0, 0.0, 0.0)  
val clusterIndex = KMeansModel.predict(obs)
```

- **ComputeCost** method returns the sum of squared distances of the observations from their nearest cluster centers. It can be used to evaluate a KMeans model.

```
val WSSSE = KMeansModel.computeCost(vectors)
```

- **Save** method persists a trained clustering model to disk. It takes a SparkContext and path as arguments and saves the source model to the given path. The saved model can be later read with the load method.

```
KMeansModel.save(sc, "models/kmean")
```

```
import org.apache.spark.mllib.clustering.KMeansModel  
val savedKMeansModel = KMeansModel.load(sc, "models/kmean")
```

Collaborative Filtering

- คือการสร้าง **recommendation** สินค้าต่างๆ โดยพิจารณาความคล้ายคลึงของข้อมูลที่เกี่ยวข้อง เช่น ถ้าลูกค้าคนแรกมีการให้ **Rating** ของสินค้าคล้ายกันกับลูกค้าคนที่สอง ระบบจะเลือกสินค้าที่ลูกค้าคนที่สองให้ **rating** ดี แต่ลูกค้าคนแรกยังไม่เคยซื้อมาแสดงให้เห็น

	item1	item2	item3	...
user1	$r_{1,1}$	$r_{1,2}$	$r_{1,3}$	...
user2	$r_{2,1}$	$r_{2,2}$	$r_{2,3}$	...
.....	...	...	...	...

User-Item Matrix

Where r_{ij} = preference score or rating of user i on item j (e.g. 1 to 5 scale)

	Item 1	Item 2	Item 3	Item 4	Item 5
User 1	0	3	0	3	4
User 2	4	0	0	2	0
User 3	0	0	3	0	0
User 4	3	0	4	0	3
User 5	4	3	0	4	4

- โดยส่วนมากใน user-item matrix จะเป็น unknown ตัว collaborative filtering ก็จะพยายามที่จะ **predict** ค่า unknown r_{ij} สำหรับ user i ต่อ item j
- เมื่อคำนวณ predicted value ออกมาได้แล้ว item j จะถูกแนะนำให้ user i โดยดูจากค่า predicted value โดยจะแนะนำให้ในกรณีที่ rating มีค่ามาก
- **Explicit feedback** คือค่า r_{ij} ที่เราทราบค่าจากการที่ user i ให้ rating มาโดยตรงหลังจากการซื้อหรือใช้ item j โดย user จะถูกถามว่าจะให้ rating เท่าไหร่
- **Implicit feedback** คือค่า r_{ij} ที่เราทราบค่าโดยการอนุมานจากระบบ ผ่านการ monitor action ของ user เช่น ประวัติการซื้อ, เคยเข้าเว็บไหนบ้าง, ใช้เวลากับเว็บไหนหลายๆ หรือ ไป follow user คนไหน implicit feedback ก็จะเป็นการลดภาระของ user ที่จะต้องมากดให้ rating แต่ความแม่นยำก็จะต่ำกว่า

Collaborative Filtering จะมี 2 เทคนิค 1.) Memory Techniques 2.) Model Based Collaborative Filtering

- 1.) **Memory Techniques** จะเป็นการ predict ค่าของ r_{ij} โดยใช้ K-nearest neighboring ในการหา user อื่นๆ ที่มีลักษณะคล้าย user i หรือหา item อื่นๆ ที่มีลักษณะคล้าย item j เป็นจำนวน K ตัว จากนั้นก็เอาค่าเฉลี่ยของความพึงพอใจ หรือ rating ของเพื่อนบ้าน มาคำนวณเพื่อ predict ค่า r_{ij} โดยเราจะคำนวณ r_{ij} ได้ 2 วิธี

- a.) **User-based Collaborative filtering** >> จะเอาค่า rating ที่เรารู้ มาคำนวณค่า similarity ระหว่าง user เพื่อหา K-nearest users

แล้ว เอา **weighted average rating** จากค่า similarity ของเพื่อนบ้าน มาคำนวณค่า r_{ij} (ที่ต้อง weight เพราะถ้า similarity มากกว่า ก็จะมี weight ที่มากกว่า)

- The predicted value can be computed by taking a weighted average of ratings among K neighbors as follow:

$$r_{i,j} = r_i + \left(\sum_{u=1}^k s(u,i) * (r_{u,j} - r_u) \right) / \sum_{u=1}^k s(u,i)$$

where

r_i is the average ratings given by user i

r_u is the average ratings given by user u

$s(u,i)$ is the similarity value between user u and i

- Two most popular similarity measures are

- **Correlation-based** (Statistical Approach) e.g. **Pearson's correlation coefficient**

$$r = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^N (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^N (y_i - \bar{y})^2}}$$

- **Cosine Similarity**, it is based on **vector space**

model which views each data instance as a vector. It measures the similarity between two data instances based on the cosine value of the angle between the two vectors that represent the two data instances.

- If d_1 and d_2 are two user vectors, then

$$\cos(d_1, d_2) = (d_1 \cdot d_2) / \|d_1\| \|d_2\|$$

where $\cdot$ indicates vector dot product and $\|d\|$ is the length of vector d .



การวัดค่า similarity ที่นิยมมาก 2 ตัว คือ

Correlation-based และ **Cosine Similarity**

- b.) **Item-based Collaborative filtering** >> จะคำนวณค่า r_{ij} จาก similarity ระหว่าง items แทน similarity ระหว่าง user

- Item based จะนิยมใช้สำหรับ high load services คือ มีจำนวน user และ item เยอะๆ เพราะ neighborhood ของ item จะมีการเปลี่ยนแปลงที่ช้ากว่าเมื่อเทียบกับ neighborhood ของ user

2.) Model Based Collaborative Filtering

- MLlib จะโฟกัสที่ user based recommendation โดยการใช้ **Matrix Completion techniques** โดยเทคนิคที่นิยมที่สุดคือ **Alternating Least Square** หรือ **ALS**

- โดยเราจะมี M เป็น user-item matrix ขนาด $m \times n$ โดยส่วนใหญ่จะเป็นค่า unknown การทำ matrix completion คือการประมาณค่า unknown จากค่าที่เราทราบเพื่อ optimize loss function

$$M = UV^T$$

where

U is the matrix of $m \times k$

V is the matrix of $n \times k$

k is the rank of the model $<< \min(m,n)$

- The objective of ALS is to minimize the loss function

$$RMSE = \sqrt{\sum_{i,j} (p_{i,j} - r_{i,j})^2 / N}$$

Where

- p_{ij} is the predicted rating of user i on item j
- r_{ij} is the actual rating of user i on item j
- N is the number of known ratings

- ALS algorithm

Step 1 : Initialize matrix V by assigning the average rating for the first row, and small random numbers for the remaining entries.

Step 2 : Fix V , solve U by minimizing the RMSE function using a regularized linear least square method.

Step 3 : Fix U , solve V by minimizing the RMSE function using a regularized linear least square method.

Step 4 : Repeat Steps 2 and 3 until convergence.

เริ่มจากการกำหนด matrix V จาก average rating ในแถวแรก และที่เหลือเป็น random

number ที่เข้าใกล้ 0 จากนั้นก็ fix V แล้วหา U โดยการคำนวณ RMSE ให้มีค่าต่ำที่สุดโดยใช้วิธี regularized linear least square method จากทั้ง fix U แล้วหา V ละทำเหมือนกับ step ที่ 2 จากทั้งทำซ้ำ step2, 3 จนกว่า RMSE จะ convergence เข้าสู่จุดต่ำที่สุด

วิธีการ **train ALS model (explicit)** จะใช้ **MatrixFactorizationModel** โดยมี input argument เป็น **RDD ประเภท Rating** และ specific parameter (ซึ่งการ Factorization คือ การแตกเป็น Matrix U V ให้ได้ ซึ่งจากสูตร $U \cdot V$ จะได้ M ออกมา ซึ่งเป็น Matrix M Prime จะมีทั้งตัวที่ Know และ Unknow ซึ่งใกล้เคียงกับ M) และจะ return ออกมาเป็น matrixFactorizationModelClass ตัว parameter ประกอบด้วย number of latent (กำหนด k), number of iterations, regularization factor (L1,L2) ส่วน level of parallelism และ random seed จะเป็น optional

```
import org.apache.spark.mllib.recommendation.ALS
import org.apache.spark.mllib.recommendation.Rating
val lines = sc.textFile("data/mllib/als/test.data")
val ratings = lines.map { line =>
  val Array(user,item,rate) = line.split(',')
  Rating(user.toInt, item.toInt, rate.toDouble)
}
val rank = 10
val numIterations = 10
val mfModel = ALS.train(ratings, rank, numIterations, 0.01)
```

- The trainImplicit method can be used when only implicit user feedback for a product is available. Feedback values r_{ui} (e.g. # views of product i webpage for user u) should be normalized to the values between 0 and 1. Instead of predicting ratings, the trainImplicit wants to predict a preference p_{ui} with a confidence value of c_{ui} . Where
$$p_{ui} = \begin{cases} r_{ui} > 0 & r_{ui} > 0 \\ r_{ui} = 0 & r_{ui} = 0 \end{cases}$$
 $c_{ui} = 1 + \alpha r_{ui}$
- α is a parameter for converting a feedback value to a confidence value (starting from 1.0).
- The objective function for the machine learning is
$$\min_{U,V} \sum_u \sum_i (p_{ui} - \hat{p}_{ui})^2 + \lambda \left(\sum_u \|u\|^2 + \sum_i \|v_i\|^2 \right)$$
 λ is a regularization parameter

- The trainImplicit method takes a feedback RDD and ALS-specific parameters as arguments and returns an instance of the MatrixFactorizationModel Class. (see Yifan Hu, Yehuda Koren, Chris Volinsky, Collaborative Filtering for Implicit Feedback)

```
val rank = 10
val numIterations = 10
val alpha = 0.01 // a parameter for converting a feedback value to a confidence value
val lambda = 0.01 // regularized parameter
val mfModel = ALS.trainImplicit(feedback, rank, numIterations, lambda, alpha)
```

The predict method ใน MatrixFactorizationModel จะรับค่า user id และ product id ที่เป็น type int และ return ออกมาเป็น rating ที่เป็น type Double

โดย prediction method ในแบบต่างๆ จะรับเป็น **pairs of RDD** ของ user id และ product id และ return เป็น RDD type Rating

- The predict method in the MatrixFactorizationModel class returns a rating for a given user and product.
- It takes user id and product id, which are of type Int, and returns a rating, which of type Double.

```
val userid = 1
val prodid = 1
```

```
val predictedRating = mfModel.predict(userid, prodid)
```

```
val usersProducts = ratings.map { case Rating(user, product, rate) => (user, product) }
val predictions = mfModel.predict(usersProducts)
val firstFivePredictions = predictions.take(5)
```

Recommend Product

- recommendProducts** >> จะ recommends product ในจำนวนที่เรากำหนดให้กับ 1 user id โดยจะรับค่าเป็น user id และ number of products ที่จะ recommend เป็น argument และ return ออกมาเป็น array of rating โดยในแต่ละ rating object จะประกอบด้วย user id, product id และ predicted rating score ตัว array ที่ return ออกมานั้นจะ sort rating เป็นแบบ descending order

```
val userid = 1
val numProducts = 3
val recommendedProducts =
  mfModel.recommendProducts(userid, NumProducts)
```

- recommendProductsForUsers** >> จะ recommends top products ตามจำนวนที่เรากำหนดให้กับทุก user ที่เราจะสามารถ recommend product นี้ได้ โดยจะใช้แค่ number of products เป็น argument แล้ว return เป็น RDD ของ users และ top recommend products

```
val numProducts = 2
val recommendedProductsForAllUsers =
  mfModel.recommendProductsForUsers(numProducts)
```

- recommendUsers** >> จะเป็นส่วนกลับกันของ recommendProducts คือ จะ recommend user ในจำนวนที่เรากำหนด ด้วย product id โดยจะรับค่าเป็น product id และ number of user และ return ออกมาเป็น array of rating โดยในแต่ละ rating object จะประกอบด้วย user id, product id และ score ใน rating field

```
val productid = 2
val numUsers = 3
val recommendedUsers =
  mfModel.recommendUsers(productid, numUsers)
```

- recommendUsersForProducts** >> จะ recommend ทุก product ให้กับจำนวน top user ที่เรากำหนด ว่าใน top user นี้จะมี product อะไรที่จะ recommend บ้าง ก็จะใช้ค่า number of users แล้ว return ออกมาเป็น RDD ของ product และ top user ที่

เรา recommend

```
val numUsers = 2
val recommendedUsersForAllProducts =
  mfModel.recommendUsersForProducts(numUsers)
val ruFor4Products =
  recommendedUsersForAllProducts.take(4)
```

- The save method persists a MatrixFactorizationModel disk. It takes a SparkContext and path of the saved file as arguments.

```
mfModel.save(sc, "models/mf-model")
```

- The load method can be used to read a previously saved model from a file.
- ```
import
org.apache.spark.mllib.recommendation.MatrixFactorizationModel
val savedmfModel = MatrixFactorizationModel.load(sc, "models/mf-model")
```

## Model Evaluation

- **Regression Metrics** class can be used for evaluating models generated by regression algorithms. It can calculate mean squared error, root mean squared error, mean absolute error etc.

```
val numIterations = 100
```

```
val lrModel = linearRegressionwithSGD.train(labeledPoints,
 numIterations)
```

```
val observedandPredictedLabels = labelPoints.map {
 observation =>
```

```
val predictedLabel = lrModel.predict(observation.features)
 (observation.label, predictedLabel) }
```

```
val regressionMetrics = new
 RegressionMetrics(ObservedAndPredictedLabels)
```

```
val mse = regressionMetrics.meanSquaredError
```

```
val rmse = regressionMetrics.rootMeanSquaredError
```

```
val mae = regressionMetrics.meanAbsoluteError
```

- **BinaryClassificationMetrics** class can be used for evaluating binary classifiers. It provides methods for calculating Receiver Operating Characteristics ROC curve (TPR vs FPR curve), area under the ROC (AUC).

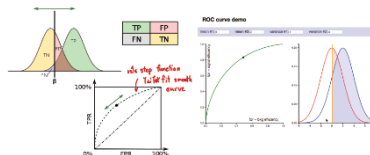
```
val svmModel = SVMWithSGD.train(trainingData,
 numIterations)
```

```
val predictedAndActualLabels = testData.map {
 observation =>
```

```
val predictedLabel =
 svmModel.predict(observation.features)
 (predictedLabel, observation.label) }
```

% predictedLabel is actually a score between 0 and 1.0) %

```
val metrics = new
 BinaryClassificationMetrics(predictedAndActualLabels)
val auROC = metrics.areaUnderROC()
```



The focus of precision is positive predictions. It indicates how many of the positive predictions are true.

$$\text{Precision} = \frac{TP}{TP + FP}$$

The focus of recall is **actual positive classes**. It indicates how many of the positive classes the model is able to predict correctly.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- For evaluating a multi-class classifiers, the `MulticlassMetrics` class provides methods for calculating precision, recall, F-Measure etc.

```
import org.apache.spark.mllib.classification.NaiveBayes
import org.apache.spark.mllib.evaluation.MulticlassMetrics
```

```
val model = NaiveBayes.train(trainingData)
```

```
val predictionsAndLabels = testData.map { d =>
 (model.predict(d.features), d.label) }
```

```
val metrics = new MulticlassMetrics(predictionsAndLabels)
val recall = metrics.recall
```

```
val precision = metrics.precision
```

```
val fMeasure = metrics.fMeasure
```

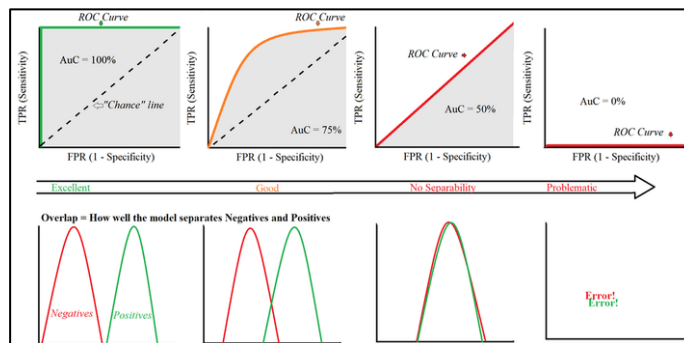
คือการประเมิน Model ที่ทำมันดีพอที่จะเอาไปใช้งานจริงหรือยังโดยพิจารณาจาก ความแม่นยำ ความถูกต้อง เมื่อเทียบระหว่างค่าจริงและค่าพยากรณ์ และต้องประเมินทั้ง Training Data และ Testing Data เช่น Regression algorithms ใช้ Model Evaluating คือ mean squared error, root mean squared error, mean absolute error และอื่นๆ โดยเปรียบเทียบระหว่าง Actual กับ Predict

## Classification ៣ ២២២

1. Binary Classification Metrics ใช้สำหรับ binary classifiers ที่มีผลลัพธ์แค่ 2 ตัวแปร เช่น positive-negative, True-False และ 0-1 เป็นต้น และสามารถคำนวณ Receiver Operating Characteristics หรือ ROC curve (TPR vs FPR curve), area under the ROC (AUC) หรือพื้นที่ใต้เส้นโค้ง

และต้องมีการกำหนด Threshold เพื่อแบ่ง Good กับ Bad

เช่น Threshold  $\geq 0.5$  ดังนั้นเมื่อ Y predict = 0.5 เมื่อเทียบกับ Threshold จะได้ว่า Y Prime = 1 เป็นต้น ถ้าดูเขาสองลูก สามารถแจกออก จากกันได้อย่างชัดเจน แปลว่าดี สามารถแยก Good/Bad ได้ดีมาก หรือ พื้นที่ภายในโค้งหยะก็แสดงว่า Model ดี



**ROC Curve (receiver operating characteristic)**

<<<<<พริ๊ดๆได้ดึ

ยิ่งดูเขาแยกยิ่งแยกได้ดีเท่านั้น

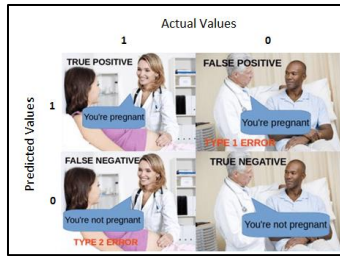
## เขาสองลูกมีแค่ binary เท่านั้น

ในการศึกษาเรื่อง ROC curve มี 2 ประเด็นสำคัญ ที่ต้องให้ความสำคัญ ได้แก่

1. **Optimal cut-off point** ของ ROC : การเลือก Optimal cut-off point หรือจุดที่อยู่บนกราฟ ที่จะได้ค่า AUC มากที่สุด (ให้ผลดีที่สุด) คือ จุดที่อยู่ใกล้มุมซ้ายบนมากที่สุด ซึ่งจะเป็นจุดที่ Sensitivity สูงและ Specificity สูงด้วยเช่นกัน

2. พื้นที่ใต้กราฟ (AUC; Area under the curve) : ค่า AUC ยิ่งเข้าใกล้ 1 มากยิ่งดี เพราะโมเดลจะมีประสิทธิภาพ

- ## 2. Multi-class classifiers ใช้ได้กับทุก Class



Confusion Matrix จากภาพข้างบนเป็นแบบ predict-actual ในภาพนี้เราจะแทนคุณหมอบอกให้เป็น model ที่ทำนายว่าคนไข้กำลังตั้งท้องอยู่หรือไม่

- 1) True Positive คือ ข้อมูลที่ทำนายตรงกับข้อมูลจริงในคลาสที่กำลังพิจารณา (โมเดลบอกท้อง ผลจริงคือท้อง)
- 2) False Positive คือ ข้อมูลที่ทำนายผิดเป็นคลาสที่กำลังพิจารณา (โมเดลบอกท้อง ผลจริงคือไม่ท้อง)
- 3) False Negative คือ ข้อมูลที่ทำนายผิดเป็นคลาสที่ไม่ได้พิจารณา (โมเดลบอกไม่ท้อง ผลจริงคือท้อง)
- 4) True Negative คือ ข้อมูลที่ทำนายตรงกับข้อมูลจริงในคลาสที่ไม่ได้พิจารณา (โมเดลบอกไม่ท้อง ผลจริงคือไม่ท้อง)

จากตัวอย่างนี้ แสดงให้เห็นว่า จะมีกรณีที่เราจำเป็นต้องพิจารณาจากสถานการณ์จริงว่าเหตุการณ์ไหนที่อยู่ในเกณฑ์ที่รับได้หรือเหตุการณ์ไหนที่ต้องระมัดระวังไม่ให้เกิดขึ้น กรณีข้างบนนี้ type 2 error น่าจะไม่ควรให้เกิด เพราะจะทำให้เกิดอันตรายได้ ดังนั้น ควรลดค่า False Negative ให้ได้มากที่สุด

|                      |  |  |                            |   |                                                                                                                  |
|----------------------|--|--|----------------------------|---|------------------------------------------------------------------------------------------------------------------|
|                      |  |  | Accuracy                   | = | $\frac{TP+TN}{TP+FP+TN+FN}$                                                                                      |
|                      |  |  | True Positive Rate         | = | $\frac{TP}{TP+FN}$                                                                                               |
|                      |  |  | False Positive Rate        | = | $\frac{FP}{FP+TN}$                                                                                               |
|                      |  |  | Recall                     | = | $\frac{TP}{TP+FN}$                                                                                               |
|                      |  |  | Precision                  | = | $\frac{TP}{TP+FP}$                                                                                               |
|                      |  |  | $F_1$ Measure              | = | $\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$                        |
|                      |  |  | $F_\beta$ Measure          | = | $\frac{(1+\beta) \times \text{Precision} \times \text{Recall}}{(\beta \times \text{Precision}) + \text{Recall}}$ |
| (a) Confusion Matrix |  |  | (b) Definitions of metrics |   |                                                                                                                  |

1. Accuracy : ค่าความถูกต้อง คำนวณได้จากสูตร  

$$\text{Accuracy} = (TP+TN)/(TP+TN+FP+FN)$$
 หรือ ค่าที่โมเดลทายถูกทั้งหมด/ค่าทั้งหมด
2. Precision หรือ Positive Predictive Value : ค่าความแม่นยำ  

$$\text{Precision} = TP/(TP+FP)$$
 หรือ ค่าที่โมเดลทายเป็นคลาสที่กำลังพิจารณาถูก/ค่าที่โมเดลทำนายว่าเป็นคลาสที่กำลังพิจารณาทั้งถูกและผิด
3. Sensitivity หรือ Recall : ความไว หาค่าได้จากสูตร  

$$\text{Sensitivity/Recall} = TP/(TP+FN)$$
 หรือ ค่าที่โมเดลทายเป็นคลาสที่กำลังพิจารณาถูก/ค่าเหตุการณ์จริงเป็นคลาสที่กำลังพิจารณาทั้งถูกและผิด
4. Specificity : ความจำเพาะ หาค่าได้จากสูตร  

$$\text{Specificity} = TN/(TN+FP)$$
5. F1-Score : เป็นค่าที่ได้จากการเอาค่า precision และ recall มาคำนวณรวมกัน (F1 สร้างขึ้นมาเพื่อเป็น single metric ที่วัดความสามารถของโมเดล ไม่ต้องเลือกระหว่าง precision, recall เพราะเฉลี่ยให้แล้ว)  

$$F1 = 2 * [(precision * recall) / (precision + recall)]$$

## ML Pipeline

- ทำงานบน DF เป็นหลัก (ไม่ได้ทำงานบน RDDs เหมือน MLLib) รวมถึงมี concept ของ pipeline ช่วยในการให้ state ต่างๆทำงานได้อย่างต่อเนื่อง //MLlib เราเหมือนต้องเขียนโค้ดเอง เริ่มตั้งแต่ pre-processing, split data ต่างๆ, train-test, evaluate คือมีหลายขั้นตอนมาก ดังนั้นเราจะเอาขั้นตอนต่างๆ ที่ทำใน MLLib นี้มาฟอร์มเป็น state ใน pipeline / workflow ได้ โดยอาจตั้งชื่อ pipeline (สร้าง object ขึ้นมา) แล้วให้ execute ตาม state ที่เรากำหนด ตั้งแต่ต้นจนจบกระบวนการเมื่อส่ง input data เข้าไป
- ML Pipelines** standardize APIs for machine learning algorithms to **make it easier to combine multiple algorithms into a single pipeline, or workflow**
- Main concept ใน ML -> ต่างจาก MLLib ยังไง**
  - DataFrame** : เก็บข้อมูลในรูป DF (input และ output ในรูป DF) (DF เป็น structure data มากกว่า RDDs และทำงานได้ optimize มากขึ้น) โดย เมื่อเก็บ data ในรูป DF ทำให้ทำงานได้เร็วขึ้น รวมถึงจัดการด้วย sparkSQL ได้ และนำมาใช้กับ ML นี้ได้โดยตรง
    - DF ประกอบไปด้วย row, column เมื่อใช้ DF กับ ML สามารถเก็บ 1 column เป็น text/stg ยาวๆเลยก็ได้ ex. การทำ text analytic 1 column อาจเป็น instance ยาวๆเลยก็ได้ โดย 1 row คือ 1 doc / 1 column เก็บ feature vector, label เก็บ อีก column, prediction ที่ได้เก็บอีก column เป็นต้น (ไม่เก็บในรูป RDDs)
  - Transformer** : เป็น operation หลัก โดยเมื่อ input data เข้าไป transform ได้ output ออกมา ใน ML มี transformer ที่ใช้ transform inputDF -> outputDF เช่น มี test data เก็บใน DF มันจะ Predict label ออกมาเก็บใน outputDF (เหมือน insert column เข้าไป สำหรับ ML model แต่ออกมาเป็น DF อันใหม่ เพราะมันเป็น immutable)
    - เป็น code / object ที่ประกอบด้วย parameter และ model ที่เรา learn ได้
    - Feature transformer อาจแปลง inputDF ให้เป็นอีกฟอร์มนึง ex input text ได้ column ใหม่เป็น feature vector ใส่ในอีก column ใน outputDF อันใหม่
    - learning model : รับ test/train data ที่จะประเมิน output เป็น DF ใหม่ที่มีการ append column เข้าไป
  - Estimator** : ใช้ในการ train model, เมื่อรับ inputDF มา output ออกมาเป็น model ex. เรามี SVM model เมื่อส่ง inputDF เข้าไปเป็น training data ตัว Estimator รับไปแล้วใช้ SVM algorithm สร้าง model transformer เพื่อนำเอาไป predict คำต่อไป
  - Pipeline** : ประกอบด้วย state แต่ละ state จะเป็น Transformer หรือ Estimator (<-ใช้ fit ในการสร้าง ML model ซึ่งซึ่งจะกลายเป็น transformer //เหมือนเข้ามาแทน estimator ที่ state นั้น แล้วรับ input เข้ามาและทำงานต่อไป) มาต่อกันเป็น ML workflow (pipeline)
    - เอา state ที่เกิดขึ้น (เช่น state ใน MLLib) มาฟอร์มเป็น pipeline โดยสร้างเป็น object ใหม่ขึ้นมา เมื่อนำ pipeline ไป execute ข้อมูล ก็จะมีการทำ state ต่างๆ ตั้งแต่เริ่มจนจบ
    - //ex.มี text data -> เราต้องมีการตัดด้วย white space เพื่อให้ออกมาเป็นคำๆก่อน -> แปลงคำให้เป็น feature vector -> เอา vector มา classification ว่าเกี่ยวกับเรื่องอะไร หา class ของ text //



- ตัวอย่างตามรูป : 3 state โดย state 1,2 คือ **transformer** และ **hashingTF 3 (logistic regression)** คือ estimator ข้างล่างจะแสดงให้เห็น data flow ของ pipeline มี raw data อยู่ใน data frame เก็บ text file ใน 1 row – row ถัดไปเป็นอีก text นึง ทั้งหมดอยู่ใน DF เดียว (เก็บใน column 1 column) โดน Tokenizer.transform() ออกเป็นคำ เมื่อตัดคำก็จะเพิ่ม column เข้าไป HashingTF.transform() จะ convert column นั้นให้เป็น feature vector ผลลัพธ์ที่เพิ่มเป็น column ใหม่ สมมติอยากรู้ว่ามีคำอะไรบ้างใน text ตัวนี้ ได้เป็น logistic regression model
- เมื่อเราจะ execute pipeline เราจะใช้คำสั่ง fit ซึ่งหมายถึง pipeline เป็น estimator ตอน train data เมื่อ fit แล้วจะเป็น transformer (ได้ pipeline model ออกมา) เมื่อใส่ test data เราจะได้ outputDF ที่มีผลลัพธ์ที่เราต้องการ
- Transformer และ Estimator จะต้องมีการใช้ parameter ที่ set เข้าไป โดยโดยมี common API ใช้ในการกำหนด parameter ของ Transformer และ Estimator ที่ใช้ด้วยกันได้
- Param = เป็นเซตของ (parameter, value) pair โดยเมื่อ execute ParamMap มันจะ set parameter ให้เรา
- 2 วิธีการ pass parameter ไปยัง Transformer และ Estimator ใดๆ
  - ใช้คำสั่ง set parameter สำหรับ instance
  - ใช้ ParamMap โดยพารามิเตอร์ใน ParamMap จะถูก override parameter previously specified via setter methods
- Parameter** : เมื่อเราจะให้ Transformer และ Estimator ทำอะไรเราต้องส่ง parameter โดยมี common API ใช้ในการกำหนด parameter ของ Transformer และ Estimator ที่ใช้ด้วยกันได้

## Text Analytics

- การทำ text analytic เป็นการวิเคราะห์ข้อมูลประเภท **unstructured data** โดยมี input เช่น **Corpus** ซึ่งคือ **collection** ของ text เช่น รีวิวบน ทวิต, comment ของ user, PDF file, word file, Document, web page ที่มี text อยู่ ฯลฯ โดยอาจใช้วิธี clustering จัดกลุ่ม text ก็ได้
  - หลักการของ text analytic คือการหา **pattern** ว่ามีการใช้และมี **sequence** ของคำอย่างไร / theme หรือ topic ของเนื้อหาใน text นั้น พูดหรือสื่อ ความหมายถึงเรื่องอะไร (อาทิ พูดถึง คน, สิ่งของ, สถานที่, ชาวเศรษฐกิจ, ชาวการเมือง หรือกระทั่งบอก sentiment ว่าเป็น positive, negative, neutral)
  - นอกจากนี้ยังสามารถหาความสัมพันธ์ระหว่างคำในประโยคเดียวกันหรือต่างประโยค / ความสัมพันธ์ระหว่าง document ด้วยกันว่ามีความสัมพันธ์เหมือนหรือ แตกต่างกันอย่างไร หรือจะจัดกลุ่มข้อมูลอย่างไร
  - **text analytic** มี **objective** เพื่อวิเคราะห์หาความหมายของข้อความและความสัมพันธ์ระหว่างคำ (ex. Tom is my friend. He lives in BKK สามารถบอกได้ว่า He คือ Tom เป็นต้น)
  - **Application** ของ text analytic
    1. **Text classification** : แยกจำแนกข้อความ เพื่อ track ว่าข้อมูลเป็น **class / category** อะไร เช่น email เป็น spam มั้ย, document categorization เป็นหมวดหมู่ยังไง เช่น แยกแยะว่าเป็นข่าวเกี่ยวกับอะไร, web page พูดถึงเรื่องอะไร – เป็น **supervise learning** เพราะต้องมี **class document** ว่าเป็น **label** อะไรแล้วจึง **train model** ออกมา
    2. **Sentimental Analysis** : ความรู้สึก หรืออารมณ์ที่สื่อในเนื้อหา (ความคิดเห็น) เช่น วิเคราะห์ความเห็นทางการเมืองของคนบนเฟสบุ๊ค, ทวิต, โซเชียลมีเดียอื่นๆ ว่าผู้เขียนรู้สึกอย่างไร + / - / เป็นกลาง ดังนั้น ต้องดูภาพรวมของ text ว่ามี mood รวมๆ ไปในทิศทางไหน – เป็น **supervise learning** เพราะต้องมี **class document** ว่าเป็น **label** อะไรแล้วจึง **train model** ออกมา
    3. **Topic Modeling** : หาว่ามี **topic** หรือ **theme** อะไรที่อยู่ใน **document** ของเรามาก (หาหัวข้อทั้งหมดที่เป็นไปได้ใน corpus), ดูว่าใน document อยู่ ใน cluster ไหน ด้วย degree เท่าไหร่ (มีหลาย topic แต่นำหนักแต่ละ topic อาจไม่เท่ากัน) โดยอาจแปลงข้อมูลในรูป text ให้เป็น vector เพื่อนำไปใช้จัดกลุ่ม clustering / classification – เป็น **unsupervised learning** เพราะเราไม่รู้ว่ามี **topic** มีอะไรบ้าง ต้องจัดกลุ่มว่ามี **topic** ใดบ้าง / มีคำอะไรปรากฏบ้าง
    4. **TF-IDF (Term Frequency – Inverse Document Frequency)** : เนื่องจากข้อมูลเป็น unstructured data เราต้องการแปลงให้อยู่ในรูปของ **vector base** เพื่อนำไปวิเคราะห์ต่อได้ โดยใช้วิธีการ TF-IDF โดย
      - **Term Frequency** : นับจำนวนคำ (ความถี่ของคำ) ที่ปรากฏอยู่ใน **document** ว่าปรากฏขึ้นกี่ครั้ง \*\*note : vocabulary = คำศัพท์ที่เป็นไปได้ทั้งหมดใน doc บางคำอาจไม่เกิดใน doc เลยก็ได้
      - แต่คำที่ **common** มาก ๆ เช่น a, an, the แต่ไม่ได้มีความสำคัญมาก เพราะใช้ในทุก doc ดังนั้น เราจะนับแค่ Term Frequency (ความถี่ของคำ) ไม่ได้ ดังนั้นจึง ต้องมี IDF ออกมา **adjust** ว่าคำนั้นมีความสำคัญมากน้อยแค่ไหน ถ้า **common** เกินไปก็ควรมีน้ำหนักลดลง
      - ดังนั้น TF-IDF จะคือนำหนักของแต่ละ term เช่น term x ใน doc y .: ถ้ามีคำที่เป็นไปได้ 100k คำ เราก็จะมี **vector** ที่เป็นไปได้ 100k คำ เราก็จะมี **weight** ของแต่ละคำสำหรับ document 1 doc
- (ในกรณีนี้เรากำหนด vocab มาแล้ว บางคำอาจไม่มีอยู่ใน corpus นี้)  
จำนวน document ทั้งหมดใน corpus
- For Spark, the following formula for IDF is used,  
$$idf_x = \log((N + 1) / (df_x + 1))$$
$$w_{x,y} = tf_{x,y} \times \log\left(\frac{N}{df_x}\right)$$

**TF-IDF**  
Term x within document y  
df<sub>x</sub> = number of documents containing x  
N = total number of documents
5. **Named Entity Recognition (NER)** : หาชื่อใน text โดย **ตรวจจับการใช้คำและคำนามในประโยคเพื่อดึงข้อมูลเกี่ยวกับบุคคล** องค์กร สถานที่ และอื่นๆ เช่น “Mark Zuckerberg เป็นหนึ่งในผู้ก่อตั้ง Facebook ซึ่งเป็นบริษัทจากประเทศสหรัฐอเมริกา” เราสามารถระบุ entities ได้สามประเภท: “บุคคล” : มาร์ก ซัคเคอร์เบิร์ก, “บริษัท” : Facebook, “ที่ตั้ง” : สหรัฐอเมริกา โดยข้อมูลสามารถอธิบายตามบริบทที่เนื้อหาของ text ได้
6. **Event Extraction** : จาก name entity เราอาจนำข้อมูลมาใช้ต่อเป็น Event Extraction โดยเอามาหาความสัมพันธ์ระหว่างคำที่เรา **detect** ได้ เราก็จะทราบถึง **actual content** ของ document หรือ พยายาม **detect** ว่ามันมีเหตุการณ์/พูดถึงเหตุการณ์อะไรใน text นี้
  - เช่น ความสัมพันธ์ของ Mark กับ Facebook คือ Mark เป็น **ผู้ก่อตั้ง** Facebook หรือ หากเราสนใจ entity Facebook ที่อยู่ใน text ทั้งหมด มันจะดึง ความสัมพันธ์ต่างๆ จาก text ออกมา (ex. เอาไปใช้ทำ complain โดยหาจาก event ที่สนใจ เช่น report – หาว่า entity “Facebook” มี event ใดเกิดขึ้นกับ “Facebook” บ้างก็ได้)
7. **Dimension Reduction, Text or document vector representation** : เป็นการ **preprocessing** ข้อมูล, เมื่อมีการแปลง TF-IDF (แปลง text เป็น vector base) โดยปกติมีลักษณะเป็นเวกเตอร์ที่ยาวมากและมีค่า 0 เยอะ (เป็น **sparse vector**) จะกินพื้นที่ และใช้เวลาในการ **process vector** ดังนั้น จึงต้องลดขนาด **dimension** (จำนวนตัวแปร) ลง แต่ยังคงความสามารถ **represent text** ได้ โดยเทคนิคการลดขนาดตัวแปร เช่น SVD, LDA, Doc2Vec (เวกเตอร์ขนาดใกล้เคียงกัน คือมี **content** ใกล้เคียงกัน เช่น มี cosine ใกล้เคียงกัน เวกเตอร์ที่แปลงควรไปในทิศทางเดียวกัน ได้ออกมาเป็นเวกเตอร์ที่มีขนาดไม่ใหญ่นัก (not TF-IDF) )
- **Spark support text analytics** ได้โดย “Spark ML” มี transformer เกี่ยวกับ text analytic ที่ช่วย transform ข้อมูลในรูป text ให้ได้ output data set ออกมา โดยที่อาจทำการสร้าง output เป็นคอลัมใหม่



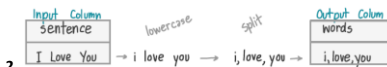
- สำหรับ text analytics มี transformer 2 แบบ

1. **Standard transformer** : ใช้ transform ปกติธรรมดา โดยหากเรามี dataset ที่เก็บในรูป DF จะให้ output เป็น DF เก็บใน column

2. **Estimator transformer** : มีการ fit และสร้าง model ขึ้นมา โดย model จะนำมาใช้ transform ข้อมูลอีกที

- **MLlib** ที่ช่วยในการทำ text analytic ในส่วนของ **pre-processing** ให้แปลง text เป็น stg ของคำให้ออกมาเป็น vector

1. **Tokenizer** : ก่อนจะวิเคราะห์เราต้องแยกคำออกเป็นคำๆก่อน ดังนั้น ต้องทำการตัดคำโดยใช้ Tokenizer และแปลงอักษรจากตัวใหญ่ให้เป็นเล็ก -> split คำออกด้วย white space, colon เป็นต้น แล้วจึงเอาคำไปนับ / convert ให้เป็น vector เพื่อนำไปใช้วิเคราะห์ต่อไป



2. **StopWordsRemover** : เป็นคำที่อาจไม่มีความสำคัญเท่าไรนัก เนื่องจากปรากฏบ่อยแต่ไม่ใช่ keyword ในประโยค เช่น a and the are we

- เรามี input DF เป็น string ของ array ที่ได้จากการตัดมาเรียบร้อยแล้ว เมื่อเจอ stop words ใน array ก็จะถูกตัดออก



4. **Ngrams** : เมื่อเราได้ sequence ของคำมาแล้ว บางครั้งเราไม่ต้องกรวิเคราะห์คำๆ เดียวโดดๆ แต่ต้องการวิเคราะห์คำรอบข้างด้วย เช่น คำที่ต่อเนื่องกัน a, b, c (2 grams) \*\*OneGram : เป็นคำๆไป

- Input DF จะเป็น array ของ Ngrams (แล้วแต่ว่าที่ n คำต่อเนื่องกัน) และเก็บ list ของ N-grams ใน output DF

5. **TF-IDF** : (ดูแค่ term และ frequency) ใช้วัดความสำคัญของคำที่ปรากฏใน corpus / collection document เมื่อเราสร้าง vector ใหม่ เช่น list ของคำ, list ของ ngrams โดยหากมีค่าเข้าใกล้ 0 น้ำหนักความสำคัญก็จะยิ่งลดลง

-  $TF-IDF = TF * IDF$

- **TF** : มี **HashingTF** เป็น **Transformer** (ต้องกำหนดจำนวนพีเจอร์ตั้งแต่ต้น เช่น มี vocab กี่คำ) ที่คำนวณ TF ให้เรา, โดยนำ set ของคำที่มีใน doc แล้วเปลี่ยนให้อยู่ในรูป vector ที่มีขนาดคงที่ เพื่อหา index ใน vector ทำให้ง่ายต่อการนับ term frequency

- **IDF** : เป็น **estimator** โดยเราต้อง fit model ก่อนเพื่อนำ model ไป transform คำนวณหา IDF ออกมา (เพราะเราต้องการทราบว่าแต่ละคำเกิดขึ้นในกี่ document (หา  $df_x$ ))

6. **Word2Vec** : (ไม่ได้ดูแค่ frequency แต่ดู co-occurent ด้วย ว่ามักเกิดคำนี้บ่อยๆใกล้ๆกัน เป็นต้น) เป็น estimator ใช้เทคนิค skip-gram เพื่อแปลงคำในประโยค / corpus ให้ออกมาอยู่ในรูป vector เรียกหลักการนี้ว่า “Neural word embedding” ซึ่งบอกถึงความสัมพันธ์ของคำด้วย หากคำมีความสัมพันธ์ vector ก็จะมีค่าคล้ายกันด้วย

- เช่น เรามี corpus ของคำ (collection ของ sentence) เกี่ยวกับสัตว์ : A dog was barking, Some cows were grazing the grass

- การใช้ neural network ที่มีเลเยอร์ที่ซ่อนอยู่ ทำให้เราสามารถเรียนรู้ได้ว่า สุนัขและเสียงเห่ามีความเกี่ยวข้องกัน วัวและหญ้ามีความเกี่ยวข้อง/สัมพันธ์กัน เพราะมีการปรากฏใกล้กันบ่อยๆ

7. **CountVectorizer** : เป็น estimator ที่แปลงจาก text doc เป็น vector นับ term frequency โดยแปลงจาก text doc ให้เป็น token counts เพื่อนำ vector ไปวิเคราะห์

- ไม่ต้องกำหนดจำนวนพีเจอร์ตั้งแต่ต้น เช่น มี vocab กี่คำ เพราะจะมีการสร้าง dictionary คำที่เป็นไปได้ทั้งหมดจาก corpus ของเรา แล้วสร้าง transformer มาเพื่อ convert / นับตาม dictionary นั้น

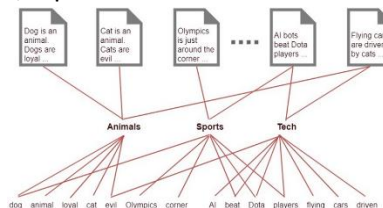
8. **Topic Modeling using LDA (Latent Dirichlet Allocation)** : เป็น estimate, แปลง doc ให้อยู่ในรูป topic ว่าจะมีน้ำหนักแต่ละ topic เท่าไร โดยแต่ละ topic ถูกกำหนดด้วยคำที่มักใช้ใน topic นั้นๆ เป็นการทำให้ soft clustering อย่างหนึ่ง (1 doc assign ได้มากกว่า 1 cluster)

- LDA เป็น unsupervised clustering algorithm โดยเราจะกำหนดว่า ต้องการกี่ topic (k topic)

- Topics ถูก represent ด้วยคำ (Topic นี้มักมีคำนี้เกิดขึ้น โดยคำแต่ละคำมีน้ำหนักในแต่ละ topics เท่าไร) เช่น Topic : ข่าวการเงิน มีคำว่าดอกเบี้ยปรากฏบ่อยๆ

- Doc แต่ละอันเมื่อ transform แล้วจะออกมาเป็น vector ซึ่งคือ distribution ของ topic ว่ามีกี่ topic (vectorขนาดk)

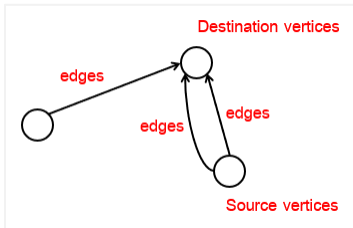
- จัดเอา doc มา cluster โดย doc หนึ่งอาจอยู่ใน cluster/topic ด้วยน้ำหนักที่ต่างกัน



- LDA : พยายามจัดกลุ่ม doc โดยใช้ statistic model โดยดูจากคำว่าปรากฏบ่อยขนาดไหน (ดูจาก word ที่ common เพื่อจัดกลุ่มเข้าด้วยกัน แต่ไม่ใช่ hard clustering แยกเป๊ะๆว่า doc อยู่ใน topic ไหน

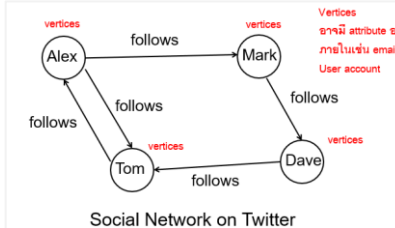
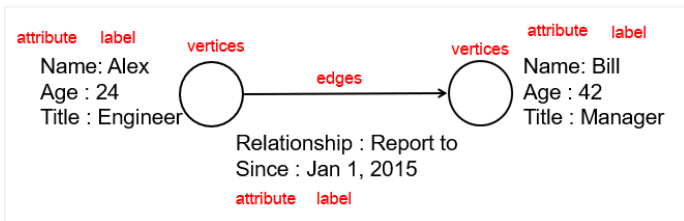
- **Text Classification** : เมื่อแปลง text เป็น vector format แล้ว เราจะทำ text classification logistic โดยใช้ logistic regression model

## GraphX



กราฟคือข้อมูลที่เป็น unstructured data ประกอบด้วย 2 ส่วนคือ vertices (node) และ edges (สิ่งที่บอกถึงความสัมพันธ์ระหว่าง 2 vertices) โดย 2 vertices นั้นสามารถมีมากกว่า 1 edges ได้จะเรียกว่า multigraph กราฟสามารถเป็นได้ทั้งแบบ directed (edges มีทิศทางทางระหว่าง vertices) และ undirected (ไม่มีหัวลูกศร)

**Property graph** คือกราฟที่มีข้อมูลรายละเอียด (attribute & label) เก็บอยู่ภายในทั้ง vertices และ edges ด้วย จัดเป็น directed multigraph เช่น กราฟบอกถึงความสัมพันธ์ของพนักงานในบริษัท, กราฟของ social network ใน platform ต่างๆ หรือ กราฟของ webpage ที่มี edges คือ link ที่เชื่อมระหว่างหน้า page



**GraphX** คือเครื่องมือวิเคราะห์ข้อมูลในรูปของกราฟ ที่จัดการข้อมูลกราฟขนาดใหญ่ และทำงานแบบกระจายบน Spark เพื่อเพิ่มความเร็ว มี Spark core API ทำงานกับภาษาอื่นได้ โดยกราฟที่ใช้ใน GraphX คือ Property graph

เครื่องมือในการ Operations ของ GraphX มีทั้ง

**1.fundamental operators** คือเครื่องเบสิคในการทำงานทั่วไป เพื่อให้นำไปใช้ในการวิเคราะห์กราฟต่างๆ ได้

**2.advanced operators** คือเครื่องมือที่มี algorithm ของ graph สำเร็จรูปให้เราเองได้โดยไม่ต้องสร้างโปรแกรมเองจาก fundamental operators เช่น page rank (google), strongly connected components และ triangle count เพื่อให้นำไปใช้ในการวิเคราะห์กราฟต่างๆ ได้

**3.Google 's Pregal API** เป็น framework หรือขั้นตอน เพื่อใช้ในการสร้าง graph algorithm เช่น implement algorithm page ranking เพื่อให้สร้าง graph algorithm ได้ง่ายมากขึ้น โดยจะทำงานแบบขนานหลายๆ node พร้อมกัน เพื่อเพิ่มความเร็วของกระบวนการทำงาน

**Process for graph analytics pipeline** ขั้นตอนในการวิเคราะห์ข้อมูลในลักษณะของกราฟ

**a) Read raw data** อาจจะมีอยู่ในรูปของ text csv, attribute of vertices & attribute of edges, ความสัมพันธ์ระหว่าง edges ที่แสดงทั้ง source, destination vertices

**b) Preprocess data** (e.g., Cleanse data)

**c) Extract vertices and edges to create a property graph** สร้างข้อมูล property graph ในส่วนของ vertices และ edges

**d) Slice a subgraph** ตัดบางส่วนของกราฟที่เราสนใจออกมาจากกราฟทั้งหมด (filter) คือดึงบาง vertices, edges เช่น ดึงแค่ 10 user ที่เราสนใจตามเงื่อนไขจาก 100 users

**e) Run a graph algorithm** นำข้อมูลจาก subgraph มารัน graph algorithm

**f) Analyze the results** ผลลัพธ์ที่ได้

**g) Repeat steps d) with another slice of the graph**

**Data type in GraphX** (data abstractions) ของข้อมูลในการวิเคราะห์กราฟมี 2 แบบ คือของส่วนที่เป็น vertices และ edges

**1.Vertex RDD** - เป็น RDD ที่เก็บข้อมูลของ vertices ใน property graph โดย 1 vertices คือ 1 element ใน RDD ข้อมูลใน Vertex RDD จะเก็บอยู่ในรูปของ key value pair โดย key คือ vertices จะมีการสร้าง id (64-bit long int) ให้ vertices นั้นๆ และ value คือ attribute ของ vertices ที่เป็นของ key vertices นั้นๆ โดย value จะเป็น data types อะไรก็ได้แต่ต้องกำหนดให้เช่น int, string, หรือระบุ types

**2.Edge** - มีลักษณะคล้ายกับ Vertex แต่จะต้องประกอบด้วย 3 ค่าคือ source vertex id, destination vertex id และ edge attribute

**Create graph** กราฟจัดเป็น class ชนิดหนึ่งที่มี method สำหรับทำการ transform หรือวิเคราะห์ข้อมูลต่างๆ class สามารถนำไปสร้างเป็น instance graph ใหม่ขึ้นมาได้โดยใช้ object Graph เรียกว่า factory method (pump graph from RDD) เพื่อสร้างกราฟจากคู่ของ RDD ระหว่าง vertices และ edges เช่นใช้ในการสร้าง property graph มีลักษณะเป็น immutable คล้าย RDD โดยจะมีการแบ่งการจัดเก็บด้วยวิธี vertex partitioning เพื่อการทำงานแบบกระจายและป้องกันการสูญเสีย

การอ้างอิงถึง 1 graph (1 instance of graph) ประกอบด้วย 2 RDD คือ

- 1.partitioned collection of vertices - เก็บ vertices ในรูปของ RDD เนื่องจากมีปริมาณมาก สามารถสร้างได้ทั้งจาก list, text file, หรืออื่นๆ
- 2.partitioned collection of edges - เก็บ edges ในรูปของ RDD เนื่องจากมีปริมาณมาก สามารถสร้างได้ทั้งจาก list, text file, หรืออื่นๆ

ตัวอย่างในการสร้างกราฟ (graph) มี 3 อย่างที่ต้องการในการสร้างกราฟคือ

1.vertexRDD 2. edgeRDD 3.default vertex property

ตัวอย่าง graph attribute (graph properties) ที่สามารถเรียกค่าออกมาได้เมื่อเราสร้าง graph ได้เรียบร้อยแล้วเช่น #vertices, #edges, #indegrees, #outdegrees

#### Number of edges – numEdges

```
val numEdges = socialGraph.numEdges
```

จำนวน edges ทั้งหมดของกราฟ

#### Number of vertices - numVertices

```
val numVertices = socialGraph.numVertices
```

จำนวน vertices ทั้งหมดของกราฟ

#### # of incoming edges - inDegrees

```
val inDegrees = socialGraph.inDegrees
```

The inDegrees is an RDD containing pairs of vertexId and its inDegree

จำนวน edges ทั้งหมดที่เข้าหา vertices นั้นๆ ออกมาในรูปของ RDD ระหว่างค่า & vertex ID

#### # of outgoing edges – outDegrees

```
val outDegrees = socialGraph.outDegrees
```

The outDegrees is an RDD containing pairs of vertexId and its outDegree

จำนวน edges ทั้งหมดที่ออกจาก vertices นั้นๆ ออกมาในรูปของ RDD ระหว่างค่า & vertex ID

```
Library graphX
import org.apache.spark.graphx._
Scalar: immutable class 2 Attribute ใน class ของ vertex
case class User(name:String, age:Int) User defined types
Id ของ vertex Value ของ vertex
val users = List((1L, User("John", 26)), (2L, User("Bill", 42)), (3L, User("Carol", 18)), (4L, User("Tom", 30)))
val userRDD = sc.parallelize(users) Vertex RDD List ที่จะต้องมี 3 ค่าคือ
val follows = List(Edge(1L, 2L, 1), Edge(2L, 3L, 1), Source, destination and Edge(3L, 4L, 1), Edge(4L, 5L, 1)) edge property
val followsRDD = sc.parallelize(follows) Edges RDD
val defaultUser = User("NA", 0) หากไม่มีการระบุ vertices จะเป็นแบบนี้เช่น vertex 5L
val socialGraph = Graph(userRDD, followsRDD, defaultUser) ใน edges ไม่มี Factory object ในตอนสร้าง
```

**Graph Operations** ชนิดของ operations ที่สามารถทำงานกับ graph ได้เมื่อมี graph มาแล้ว

- **Property Transformation** – การเปลี่ยนค่า properties ใน attribute ของ vertices หรือ edges โดยผลลัพธ์จะเป็นกราฟใหม่ที่มี properties เปลี่ยนแปลงไป เนื่องจากเป็น immutable (RDD) ที่ WORM แต่ยังคงโครงสร้าง
- **Structure Transformation** - การเปลี่ยนโครงสร้างภายใน vertices หรือ edges เช่นการเพิ่มจำนวน edges โดยผลลัพธ์จะเป็นกราฟใหม่
- **Join** - การเชื่อมกันระหว่าง 2 graph โดยผลลัพธ์จะเป็นกราฟใหม่
- **Aggregation** - การนำค่าภายใน vertices หรือ edges มา aggregation กัน
- **Graph-parallel computation** - Google 's Pregel API
- **Graph Algorithm** - advanced operators

### Property Transformation Operators

**mapVertices** – เป็น method ในการเปลี่ยนข้อมูล properties ภายใน Vertices (ทั้งหมด/บางส่วน) ด้วย map function เดียวกันทั้งหมด แต่การเปลี่ยนนั้นจะขึ้นอยู่กับค่าเดิมของ vertex ID นั้นๆ ด้วย ผลลัพธ์จะได้กราฟใหม่ transform(old graph) → new graph ที่มีจำนวน attribute ใน vertex ID เท่าเดิมแต่ค่าภายในเปลี่ยนไป

```
Function ทำทุก vertexID
val updateAges = socialGraph.mapVertices((vertexId, user) => User(user.name, user.age + 1))
โดยมีการเปลี่ยนที่ value ของ vertex = user เท่านั้น
ในนี้เปลี่ยนที่ attribute ages
```

ตัวอย่างเช่น เปลี่ยนค่า user.age ให้มีค่า +1 ในทุก vertex ID โดยที่ค่าใน user.name จะมีค่าเท่าเดิม โดยผลลัพธ์ที่ได้จะเป็นกราฟใหม่ชื่อ updateAges  
transform(socialGraph) → updateAges

**mapEdges** – เป็น method ในการเปลี่ยนข้อมูล properties ภายใน edges (ทั้งหมด/บางส่วน) ด้วย map function โดยจะแก้ไขได้แค่ property ของ edges เท่านั้น ไม่สามารถแก้ไข source vertex ID และ destination vertex ID ได้

```
val followGraph = socialGraph.mapEdges((e) => "follows")
เปลี่ยนจาก int >> str
```

ตัวอย่างเช่น เปลี่ยนค่า ตัวเลข 1 ใน property ของทุก edges ให้เป็นคำว่า "follows"

**mapTriplets** – เป็น method ในการเปลี่ยนข้อมูล properties ของ edges แต่สามารถอ้างอิงข้อมูลในการเปลี่ยนแปลงได้ทั้ง 3 ส่วนคือ properties ของ source vertex ID และ destination vertex ID ที่เป็นของ edge นั้นๆ รวมถึงค่าใน edges เองด้วย

```
Edge triplet ตั้งอะไรก็ได้
val weightGraph = socialGraph.mapTriplets {t =>
 if (t.srcAttr.age >= 30)
 Or desAttr 2 else 1 }

```

ตัวอย่างเช่น เปลี่ยนค่าใน edge ที่มี property ของ source vertex id ที่มี age >=30 ให้เป็น 2 ถ้าน้อยกว่าให้มีค่าเป็น 1 โดยจะเป็นค่าใหม่ใน edge จะทำกับทุก edges ในกราฟ

## Structure Transformation Operators

 เช่นการเปลี่ยนแปลงจำนวน vertices หรือ edges

**Reverse** – เป็นการเปลี่ยนทิศทางของ edges และผลลัพธ์จะเป็นการสร้างกราฟใหม่เช่น จาก vertex1 → vertex2 กลายเป็น vertex2 → vertex1

```
val reverseGraph = socialGraph.reverse
```

**Subgraph (Slide a subgraph)** – ต้องการแบ่งส่วนของกราฟทั้งหมด เพื่อวิเคราะห์แค่สิ่งที่สนใจหรือเรียกว่าการ filter จะต้องมี 2 function สำหรับกรอง vertices และ edges โดยผลลัพธ์จะเป็นการสร้างกราฟใหม่ที่มีแค่ส่วนที่ต้องการตามที่กำหนด

```
val subgraph = weightedGraph.subgraph (property ของ edge เงื่อนไข 1
Triplet ทำให้เราเข้าถึง (edgeTriplet => edgeTriplet.attr > 1,
ข้อมูล 3 ส่วนได้ (vertexId, vertexProperty) => true)
Vertices ไม่กรองเอาหมด กรองแต่ edges
predicates = function ที่ return true / fault
```

**Mask** – เป็น input structure graph เพื่อดึง subgraph ออกมาจาก source graph โดยการ match ในส่วนที่มี vertices และ edges ตรงกันออกมาและ properties จะเป็นของ source graph โดย vertices จะดูที่มี vertex ID เดียวกัน และ edges จะดูที่มี source และ destination vertex ID เดียวกัน หรือในแง่ของโครงสร้างของกราฟ

```
สร้าง input graph
val femaleConnections = List(Edge(2L, 3L, 0), Edge(3L, 1L, 0), Edge(3L, 4L, 0))
val femaleConnectionRDD = sc.parallelize(femaleConnections)
val femaleGraphMask = Graph.fromEdges(femaleConnectionRDD, defaultUser)
สร้าง graph จาก edges โดย vertex จะอิงจากคู่ของ edges
val femaleGraph = socialGraph.mask(femaleGraphMask)
Source graph input graph
```

ตัวอย่างเช่นสร้าง input graph femaleGraphMask มาเพื่อ match กับ socialGraph ให้ได้กราฟใหม่คือ femaleGraph โดยการ mask นั้นควรมี vertices ที่ vertex ID 1, 2, 3, 4 และ คู่ของ edge ตาม femaleGraphMask แต่ properties ของ edges ต้องอ้างอิงจาก socialGraph (source graph) ไม่ได้มีค่าเป็น 0

**groupEdges** – เป็นการรวม edges ที่มีหลาย edges จาก source และ destination vertex ID เดียวกันรวมเป็น 1 edge

```
val multiEdges = (Edge(1L,2L,100), Edge(1L,2L,200), Edge(2L,3L,300),
Edge 2L,3L,400), Edge(3L,1L,200), Edge(3L,1L,300))
val multiEdgeRDD = sc.parallelize(multiEdges) สร้าง RDD
val defaultVertexProperty = 1 สร้างกราฟจาก edges เอา vertexID จาก edges
val multiEdgeGraph = Graph.fromEdges(multiEdgeRDD, defaultVertexProperty)
val singleEdgeGraph = multiEdgeGraph.groupEdges(
(edge1, edge2) => edge1 + edge2)
Aggregate ค่าใน edges เดียวกัน
```

ตัวอย่างผลลัพธ์จะเหลือเพียง 1 edge ระหว่างคู่ของ vertex ID

1L → 2L = 300

2L → 3L = 700

3L → 1L = 500

**Join Operators** การเชื่อมกันระหว่าง 2 graph โดยทำการเปลี่ยนแปลงค่า properties หรือเพิ่ม properties โดยผลลัพธ์ที่ได้จะเป็นกราฟใหม่

**joinVertices** – เป็นการเชื่อมข้อมูลระหว่าง input RDD ที่เป็น collection of vertices กับ source graph โดยจะทำการการ match ค่าที่มี vertex ID เดียวกันเพื่อทำการนำค่า properties ของ vertex ที่อยู่ใน vertex ของ input มาทำการเปลี่ยนแปลงค่าใน vertex ของ source graph โดยค่าที่ไม่ถูก match จะเป็นค่าเดิมตาม source graph

โดยจะต้องมี 2 argument ในการใช้งาน function นี้คือ

- 1.RDD of pairs between vertex ID and vertex data (properties) โดย 1 vertex ควรมี 1 element ใน RDD
- 2.user defined function เพื่อใช้ในการ update vertex ใน source graph เพื่อให้เป็น vertex ตาม input โดยจะรับ triplet คือ (id ที่ match, ค่าเดิม, ค่า update)

```
Input RDD
val correctAges = sc.parallelize(List((3L,28), (4L,26)))
val correctGraph = socialGraph.joinVertices(correctAges) Argument 1
((id, user, correctAge) => user(user.name, correctAge))
Properties ใน socialGraph Input RDD
อะไรที่ math ถึงจะ apply function ด้านหลัง
```

ตัวอย่างเช่น update อายุของ user ใน socialGraph (source graph) ด้วย RDD correctAges (input) ด้วยค่า properties ของ vertex ID ที่เหมือนกัน จะได้กราฟใหม่

**\*\* ทำแค่ที่ match อันไม่ match จะอิงตามค่าเดิม \*\***

**OuterJoinVertices** – เป็นการเพิ่ม properties ใหม่ใน source graph จาก input RDD หรือการเพิ่มอีก attribute (filed) ใหม่ใน vertex โดยจะทำการกับทุก vertex ID ใน source graph และค่าที่ไม่ match จะมี properties เป็น None ใน user defined function argument ที่ 3

โดยจะต้องมี 2 argument ในการใช้งาน function นี้คือ

1.RDD of pairs between vertex ID and vertex data (properties) โดย 1 vertex ควรจะมี 1 element ใน RDD

2.user defined function เพื่อใช้ในการ update vertex ใน source graph เพื่อให้เป็น vertex ตาม input โดยจะรับ triplet คือ (id ที่ match, ค่าเดิม, ค่าใหม่)

```
case class UserWithCity (name : string, age : int, city : string)
val userCities = sc.parallelize(List((1L,"Boston"), (3L,"New York"),
(5L,"London"), (7L, "Bombay"), (9L, "Tokyo"), (10L, "Palo Alto")))
val socialGraphwithCity = socialGraph.outerJoinVertices(userCities)
((id, user, cityOpt) => cityOpt match
Argument 1
Argument 2: triplet
{ case Some(city) => UserWithCity(user.name, user.age, city)
case None => UserWithCity(user.name, user.age, "NA") })
ค่า vertex ที่ไม่มีใน RDD จะเป็น none และให้ update เป็น NA
```

ตัวอย่างเช่น เพิ่ม filed city เข้าไปใน socialGraph (source graph) และได้กราฟ socialGraphwithCity ใหม่

**\*\* ทำทุก vertex ID อันที่ไม่ match จะมีค่าเป็น None \*\***

## Aggregation Operators

**aggregateMessages** – เป็นการทำงานกับ message ที่ส่งเข้า – ออกจาก vertices และได้ผลลัพธ์เป็นค่า 1 ค่า

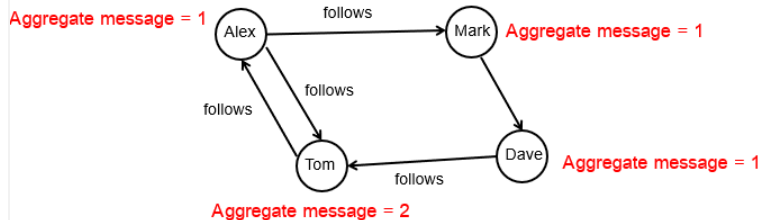
โดยจะต้องมี 2 function ในการใช้งานคำสั่งนี้กับทุก triplet คือ

1.EdgeContext เป็น class หนึ่งที่ทำให้สามารถเข้าถึงข้อมูลทั้ง 3 ส่วนได้คือ properties ของ source vertex ID และ destination vertex ID ที่เป็นของ edge นั้นๆ รวมถึงค่าใน edges เองด้วยเพื่อทำการรวบรวมการส่ง message และมีคำสั่งให้ส่ง message ไปที่ source, destination vertex ที่เป็นของ triplet นั้นๆ

2.merge/combine เป็น function ที่ทำการรับ 2 message arguments เพื่อคำนวณ message เป็นคู่ๆ และส่งกลับมาเป็นผลลัพธ์ 1 ค่าอยู่ในรูปของ RDD ที่เป็นคู่ระหว่าง (vertex ID, aggregates message value)

```
val followers = socialGraph.aggregateMessages[Int]
(edgeContext => edgeContext.sendToDst(1),
Function 1 ให้ส่งค่าเลข 1 ไปที่ des ของ triplet (สามารถเอาค่าจาก
source, des มาคำนวณแล้วส่งก็ได้)
(x,y) => (x + y))
Function 2 ในการ aggregate เป็นคู่ๆ
val followers = socialGraph.inDegrees ถ้าส่ง message คือ 1 ก็คือ indegrees
```

ตัวอย่างการส่ง message = 1 ของทุก triplet ไปทุก destination เพื่อไปทำ aggregate function (add) บน vertices หรือการหาจำนวน follower ทั้งหมดของ user นั้นๆ

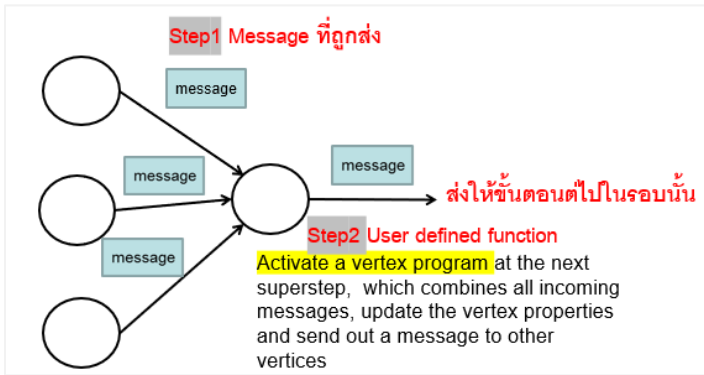


**Graph-parallel computation (Google 's Pregel API)** เป็นขั้นตอนการทำงานต่อเนื่อง และมีการทำงานหลายรอบจนถึงเงื่อนไขในการหยุด หรือค่าใน vertex เริ่มคงที่เนื่องจาก convergence หา solution แล้ว เพื่อการสร้าง Graph Algorithm ได้ง่ายขึ้น โดยจะทำงานแบบขนานพร้อมกัน เพื่อเพิ่มความเร็วในการทำงาน

**Pregel** – เป็น algorithm ที่อ้างอิงมาจาก BSP Model (Bulk Synchronous Parallel) คือ framework ในการทำงานแบบขนาน โดย Pregel จะทำงานหลายรอบ แต่ละรอบจะทำงานกับทุก node และเริ่มต้นรอบใหม่พร้อมกันเมื่อทุก node เสร็จแล้ว

**Supersteps** คือ 1 รอบของการทำงาน (1 iteration) โดยแต่ละรอบจะทำงานดังนี้





**Step1** vertex จะได้รับ message (อาจจะเป็น int, structure, key-value pair, triplet, หรืออะไรก็ได้) จากรอบก่อนหน้า หากไม่ได้รับก็จะไม่ได้ทำงานอะไร

**Step2** เมื่อได้รับ message จะทำงาน user defined function (vertex program) โดยจะทำงานพร้อมๆ กันแบบขนานในทุก vertex มีขั้นตอนดังนี้

- aggregate message โดย vertex program
- update ค่า properties ใน vertex นั้นใหม่ด้วย vertex program นั้นๆ
- ส่ง หรือไม่ส่ง message ต่อให้ขั้นตอนถัดไปในรอบนั้น อาจเป็นเดินหน้า หรือ ย้อนหลังก็ได้

**Step3** รอจนทำเสร็จครบทุก vertex และทำวนซ้ำแบบเดิมใน Supersteps ถัดไป

**Step4** สิ้นสุดการทำงานเมื่อไม่มี message ส่งให้ vertex หรือกำหนดจำนวนรอบ

**GraphX implements Pregel method** รับ 2 set of argument ในการทำงาน และให้ผลลัพธ์สุดท้ายเป็นกราฟ

first set: มี 3 arguments [parameter ในการทำงานของ Pregel]

second set: มี 3 functions ที่ใช้ใน vertex program

- Initial message ระบุ message เริ่มต้นให้กับทุก vertex ในรอบแรกใหม่ใน vertex
- Maximum number of iterations จำนวน max(supersteps) property ภายใน
- Direction ตาม/ย้อน ตามลูกศรของ edges

- 1(2). user – defined update vertex function ค่า property
- 2(3). user – defined send function (optional) อิงจากค่า
- 3(1). user – defined aggregate function

user – defined update vertex function [update ค่า property ใหม่ใน vertex]

ใน function นี้จะรับ 3 arguments (เหมือนใน triplet) คือ

- Vertex ID
- Current properties of the vertex – ค่า properties เดิม
- Message ที่ aggregate function เรียบร้อยแล้ว

ตัวอย่างในการเรียก Pregel ทำงาน

page

```
Parameter เริ่มต้น 3 argument
val firstMessage = 0.0
val iterations = 20
val edgeDirection = EdgeDirection.out ตามทิศลูกศร ส่งกลับ.in

val influenceGraph = InputGraph.pregel(firstMessage, iterations, edgeDirection)
((a,b,c) =>, // updateVertex id = a, ค่าเดิม = b, ค่าใหม่จาก agg = c
triplet =>, // sendMsg
(x,y) =>) // aggregateMsgs
3 function
```

ตัวอย่างการ implement page rank จาก Pregel เพื่อหา score ของแต่ละ

```
Algorithm 1 PageRank
input: $G : \text{Graph}[V, E]$
while $err \geq \epsilon$ do
 for vertex i do
 $R[i] = 0.15 + 0.85 \sum_{j \in N_{in}(i)} M[j]$
 $M[i] = R[i] / |N_{out}(i)|$
 Send $M[i]$ to all $N_{out}(i)$
 end for
 $err = |R - \text{previous } R|$
end while
```

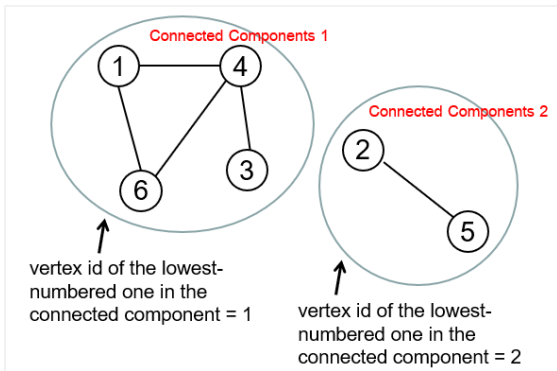
**Graph Algorithm (advanced operators)** สามารถ implement ได้จากทั้ง aggregate message (single step) และ Pregel operation (iterative) ใน GraphX มี built-in บาง algorithm ที่ใช้บ่อยๆ ให้ แต่สามารถเขียนขึ้นมาเองได้เหมือนกัน

**PageRank** – การหาความสำคัญของ vertices ในกราฟ สามารถทำงานได้หลายงานเช่น webpage (link ที่เชื่อมระหว่าง page), social network (จำนวน followers ของ user) หรือ document (reference ถึง paper) และผลลัพธ์จะเป็น RDD (vertex ID, score) โดย score ของ PageRank จะขึ้นกับ 2 ปัจจัยคือ

1. จำนวน link ที่เข้าหา webpage 2. คะแนนที่ส่งมาให้แต่ละ link นั้นจะขึ้นกับจำนวนของ link ทั้งหมดที่ออกไปจาก webpage แต่ละอันด้วย

เช่น หาก a ส่งออกไป 5 node ทำให้คะแนนของ a ที่ส่งออกไปนั้นมีค่าน้อยลงด้วยเนื่องจากการส่งออกไปหลาย node ต้องมีการเฉลี่ย score ในการส่ง

**connectedComponents** - ใช้กับ undirected graph ไม่สนใจทิศทางของ edges โดยจะเป็น subgraph ที่มีเส้นทางเชื่อมกันได้ทุก vertices ภายในกราฟๆ นั้น



สามารถลากเส้นจาก node x → node y ได้ทั้งหมด

ทั้งหมดคือ 1 กราฟ แต่แบ่งออกเป็น 2 connectedComponents ที่มี vertex ID ดังนี้

connectedComponents 1: 1, 4, 6, 3

connectedComponents 2: 2, 5

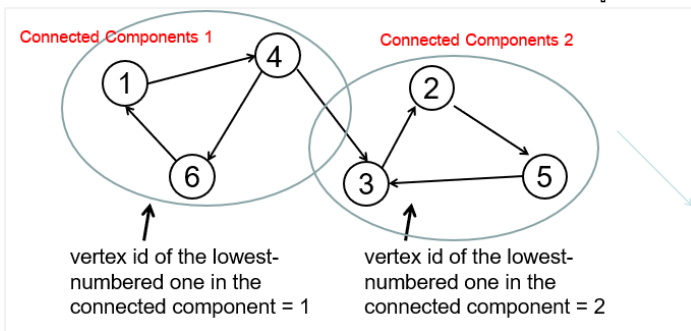
โดย connectedComponents ID จะดูจาก vertex ID ที่มีค่าต่ำสุดใน

connectedComponents นั้นๆ ผลลัพธ์จะเป็นกราฟ แต่จะมี properties เป็นค่าของ

connectedComponents ID เพิ่มมา

```
val connectedComponentsGraph = inputGraph.connectedComponents()
```

**stronglyConnectedComponents (SCC)** - ใช้กับ directed graph สนใจทิศทางของ edges โดยจะเป็น subgraph ที่มีเส้นทางเชื่อมกันได้ทุก vertices ภายในกราฟๆ นั้น แต่จะสามารถมีได้เพียง 1 เส้นทางตามหัวลูกศรเท่านั้นของ vertices แต่ละคู่



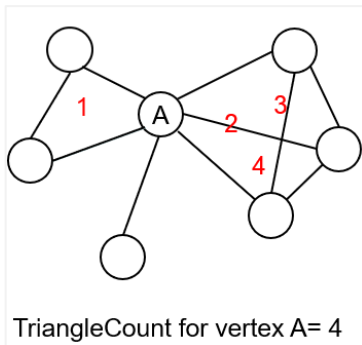
สามารถลากเส้นจาก node x → node y ได้แต่มีเพียง 1 เส้นทางเท่านั้น

โดย 4, 3 จะไม่ถือว่าเป็น stronglyConnectedComponents เนื่องจาก 3

ไม่สามารถเดินทางไปที่ 4 ได้

```
val sccGraph = inputGraph.stronglyConnectedComponents()
```

**trianaleCount** – เป็นการนับว่า vertices นั้นๆ เป็นส่วนหนึ่งในสามเหลี่ยมกี่ครั้ง



```
val trianaleCountGraph = inputGraph.trianaleCount()
```

จะให้ป็นกราฟจำนวน trianaleCount ที่ vertices อยู่

นำไปใช้ในการดูความสำคัญของ vertices นั้นๆ หรือหาผู้ที่มีความสำคัญมาก (strong community) ใน social network

$$Clustering\ Coefficient = \frac{Triangle\ Count}{\binom{d}{2}}$$

Where d is the degree or # of neighboring vertices of the vertex

ใช้ในการคำนวณหา clustering coefficient

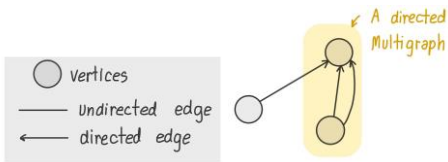
d(A) = 6

trianaleCount(A) = 4

dC2 = trianaleCount สูงสุดที่จะเป็นไปได้

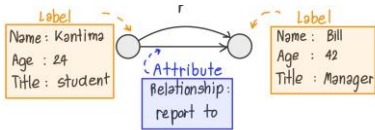
## Graph

• **Graph** เป็น Unstructured data ที่ประกอบด้วย Vertices และ Edges โดย Graph มีทั้งที่เป็น Directed (Edge มีลูกศรบอกทิศทาง) หรือ Undirected (ไม่มีลูกศร)



• **Directed Multigraph** คือ directed graph ที่มี vertices อย่างน้อย 1 คู่ที่เชื่อมต่อกันด้วย Edges ตั้งแต่ 2 Edges ขึ้นไป

• **Property Graph** คือ Directed Multigraph ที่ไม่ได้มีแค่ Vertices หรือ Edges เท่านั้น แต่มี Data พ่วงเข้ามาด้วย โดยเราเรียก data ที่พ่วงเข้ามาว่า Attributes กับ Labels



• ยกตัวอย่างกราฟ เช่น Social Network graph on Twitter or Facebook algorithm เช่น Page rank ก็ based on Google's Pregel API

## GraphX

- ข้อมูลกราฟสามารถเก็บได้หลายแบบ อาจจะเป็นกราฟอันเดียวใหญ่ๆ หรืออาจจะเก็บกราฟเล็กๆหลายๆกราฟ ใน Instant (1 instant 1 graph)
- เนื่องจากกราฟมีขนาดใหญ่ จึงต้องมีการจัดเก็บบนระบบ *Distributed* กระจายเก็บโดยใช้ *Spark* เป็น Platform เพื่อให้ process ได้รวดเร็วขึ้น
- *การทำงานแบบขนาน (Distributed)* คือการทำงานต่างๆบนแต่ละ nodes ไปพร้อมๆกัน เช่น มี 1 ล้าน vertices จะต้องแบ่งเก็บตาม nodes ในรูป RDD กระจายการ process เพื่อทำงานเสร็จเร็วขึ้น
- มี Spark Core API เป็นตัวหลัก ซึ่ง GraphX จะ on Top บน Spark core

• **GraphX มี Operators 3 แบบ คือ**

1) **Fundamental graph operators** \*กราฟสำเร็จรูปเรียกใช้ได้เลย\*

2) **Advanced operators** ที่จะ implement graph algorithms ให้เราเลย โดยจะทำงานกราฟคล้ายๆกับ Operator ที่ทำงานเฉพาะอย่างให้ โดยที่เราไม่ต้องเขียน Code เอง แต่จะเป็นคำสั่งหรือ Algorithm สำเร็จรูปมาให้ เราแค่เพียงเรียก Operator ขึ้นมาใช้งาน เช่น Page Rank (ของ Google), strongly connected components and triangle count.

3) **Google's Pregel API** เป็น Framework ที่มีขั้นตอนในการ process graph เพื่อใช้ในการ implement graph algorithm เช่น Page rank ก็ based on Google's Pregel API

## Create a Graph

```
import org.apache.spark.graphx._
```

```
case class User(name:String, age:Int)
```

```
val users = List((1L, User("John", 26)), (2L, User("Bill", 42)), (3L, User("Carol", 18)), (4L, User("Tom", 30)))
```

```
val userRDD = sc.parallelize(users)
```

```
Vertices [L]
```

```
① John, 26
```

```
② Bill, 42
```

```
③ Carol, 18
```

```
④ Tom, 30
```

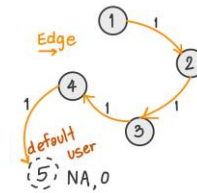
```
val follows = List(Edge(1L, 2L, 1), Edge(2L, 3L, 1), Edge(3L, 4L, 1), Edge(4L, 5L, 1))
```

```
val followsRDD = sc.parallelize(follows)
```

```
val defaultUser = User("NA", 0)
```

```
val socialGraph = Graph(userRDD, followsRDD, defaultUser)
```

โดย defaultUser กำหนดขึ้นมาในกรณีที่ยังไม่มีถูก assign properties



## Attribute ที่เราสามารถดึงออกมาได้

1. No of edges – numEdges
2. No of vertices - numVertices
3. No of incoming edges – inDegrees  
เก็บในรูป RDD เป็นคู่ๆ (vertexId and its indegree)
4. No of outgoing edges – outDegrees  
เก็บในรูป RDD เป็นคู่ๆ (vertexId and its outdegree)



## Graph analytics pipeline 7 steps:

- a) Read raw data
- b) Preprocess data (e.g. cleanse data)
- c) Extract vertices and edges to create a property graph.
- d) Slice or filter a subgraph.
- e) Run a graph algorithm.
- f) Analyze the results.
- g) Repeat steps e) to d) with another slice of the graph.

## Data Type in GraphX API:

1. **VertexRDD**: collection of vertices in a property graph.

(1 Vertex 1 element) โดยจะเก็บเป็น Key-Value pair

- **Key** is a unique id เรียก "VertexId" data type คือ 64 bit long Int

- **Value** is the data associated with a vertex (**Attribute&Label**)

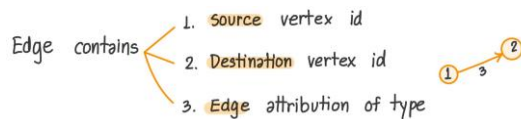
เป็น type อะไรก็ได้

- **VertexRDD[VD]** คือ Type ของ data เช่น Int, long, Double, String หรือแล้วแต่ user จะกำหนด



2. **Edge class**: เก็บ a directed edge in a property graph.

ประกอบด้วย 3 ส่วนคือ 1) Source vertex id, 2) Destination vertex id, 3) edge attributes of type เช่น Int, long, Double, String หรือแล้วแต่ user จะกำหนด



3. **Graph**: คือ class เพื่อสร้าง property graphs ซึ่งกราฟอยู่ในรูป RDD โดย instance of graph มีคุณสมบัติเหมือน RDD คือ immutable (WORM ไม่สามารถแก้ไขได้), distributed (มีการกระจายกันเก็บข้อมูล) and fault tolerant (ถ้ามีปัญหาที่ Node ก็สามารถ recover กลับมาได้)

- **Graph instance** ประกอบด้วย 2 RDD ย่อย คือ

1) collection of vertices, 2) collection of edges

- Graph class ใช้สำหรับ transform and analyze vertices and edges

- GraphX จะมี object ที่ชื่อว่า "Graph" ใช้สำหรับสร้างกราฟใหม่จาก RDD ด้วยวิธีที่เรียกว่า "Factory method" ก็คือจะบีบกราฟใหม่ โดย based on vertices and edges เดิม แล้วก็สร้าง RDD เพื่อมาเก็บ vertices and edges

## Graph Operators

1. **Property Transformation**: เปลี่ยนแค่ property เช่น ค่าต่างๆใน Vertices/Edges ผลลัพธ์ออกมาเป็นกราฟใหม่ที่ Property เปลี่ยนไป แต่โครงสร้างเดิม

1) **MapVertices**: เปลี่ยน Property ใน Vertices ...val updateAges =

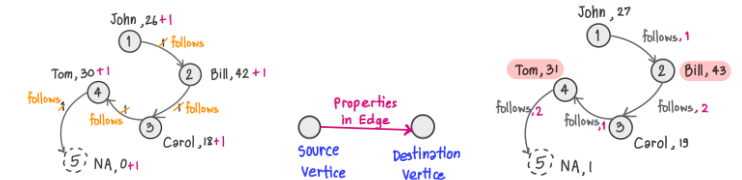
```
socialGraph.mapVertices((vertexId, user) => User(user.name, user.age + 1))
```

2) **MapEdges**: เปลี่ยน Property ใน Edges

```
val followGraph = socialGraph.mapEdges((e) => "follows")
```

3) **MapTriplets**: เปลี่ยน Property in edges โดยการนำค่าใน Vertices หัว-ท้าย เพื่อมาคำนวณหา Edge ใหม่ ... val weightGraph = socialGraph.mapTriplets { t =>

```
if (t.srcAttr.age >= 30) 2 else 1 }
```



2. **Structure Transformation**: เปลี่ยนโครงสร้างไปเลย เช่น เพิ่ม/ลด จำนวน Vertices/Edge

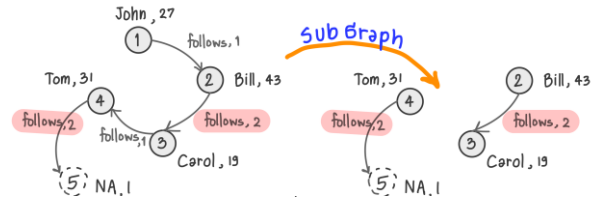


1) **Reverse**: เปลี่ยนทิศทางลูกศรของทุก Edges

2) **SubGraph**: filter เอาเฉพาะ Vertices/Edges ที่ตรงตามเงื่อนไข โดย function จะ return ค่าออกมาเป็น Boolean (T/F) วิธีการคือจะดู 2 predicates คือ predicate แรกจะ match กับ EdgeTriplet ส่วน predicate ที่สอง match กับ vertexID & vertex Property

```
val subgraph = weightedGraph.subgraph (edgeTriplet
```

```
=> edgeTriplet.attr > 1, (vertexId, vertexProperty) => true)
```



3) **Mask**: ใช้สร้าง Subgraph ใหม่ ที่ยังคงมี properties มาจาก Source graph เดิม

- ดึงเฉพาะกราฟที่ Match กันทั้ง 3 ตัว (SourceVertex, DestinationVertex, Properties in Edge) ตาม input graph โดยจะได้เป็น Returned graph ที่ดึง properties มาจาก Source graph

4) **GroupEdges**: นำค่า(Property)ใน edge ที่ขนานกัน มาคำนวณได้เป็นค่าใหม่ ทำเป็นคู่ๆจนได้ค่าเดียวใน output graph

```
val multiEdges = (Edge(1L,2L,100), Edge(1L,2L,200), Edge(2L,3L,300), Edge(2L,3L,400), Edge(3L,1L,200), Edge(3L,1L,300))
```

```
val multiEdgeRDD = sc.parallelize(multiEdges)
```

```
val defaultVertexProperty = 1
```

```
val multiEdgeGraph = Graph.fromEdges(multiEdgeRDD, defaultVertexProperty)
```

```
val singleEdgeGraph = multiEdgeGraph.groupEdges((edge1, edge2) => edge1 + edge2)
```



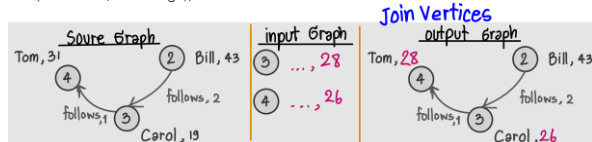
**3. Join Operators** คือการ update ค่า property เดิมที่มีอยู่ หรือเป็นการเพิ่มค่า property ใหม่เข้าไปใน vertex ซึ่งใน GrapX เราสามารถทำ Many Join Operators ได้

1) **JoinVertices**: นำค่า Collection of vertices จาก input graph ไป update ค่า properties ใหม่ใน source graph

- ใช้ 2 Arguments อันแรกคือ RDD of pairs (VertexID&VertexProperty) อันที่ 2 คือ User-specified function โดยจะ update ค่า vertex ใน source graph ถ้าตัวไหนที่ไม่ match กับค่าใน input graph ก็จะไม่ถูกเปลี่ยน

```
val correctAges = sc.parallelize (List((3L,28), (4L,26))
```

```
val correctGraph = socialGraph.joinVertices(correctAges)((id, user, correctAge) => user(user.name, correctAge))
```



2) **OuterJoinVertices**: ใช้ update current attributes หรือ Add ค่า attributes ใหม่ในแต่ละ vertex ใน Source graph

- ใช้ 2 Arguments อันแรกคือ RDD of pairs (VertexID&VertexProperty) อันที่ 2 คือ User-specified function โดยรับค่า Triplet จาก input RDD (1.vertexID, 2.current vertex attributes, 3.Optional attributes [อาจเป็นค่า None ก็ได้]) เอาไป add เพิ่มเป็น New attributes ของ vertex ใน source graph

- ถ้าใน inputRDD ไม่ได้ถูก update มันจะรับค่า none มา

- input RDD จะต้อง มี 1 ค่าต่อ 1 vertex เท่านั้นในการ join

- สมมติอยากเพิ่ม attribute ใหม่ที่ชื่อว่า City เดิมมีแค่ชื่อกับอายุ

```
case class UserWithCity (name : string, age : int, city : string)
val userCities = sc.parallelize(List((1L,"Boston"), (3L,"New York"),
(5L,"London")))
val socialGraphWithCity = socialGraph.outerJoinVertices(userCities)
((id, user, cityOpt) => cityOpt match
{ case Some(city) => UserWithCity(user.name, user.age, city)
case None => UserWithCity(user.name, user.age, "NA") })
```

#### 4. Aggregation Operators

- aggregateMessages เป็นการ aggregate ค่าจากแต่ละ vertex ใกล้เคียงที่เชื่อมกันผ่าน edges โดยมี 2 functions ที่จะคำนวณค่าในทุกๆ Triplet ใน property graph 1) **EdgeContext (Send Messages)**: ที่เรารู้คือ edge มีหน้าที่เชื่อมทั้ง 2 vertex เข้าด้วยกัน และสามารถเข้าถึง property ใน vertex ทั้งหัวและท้ายได้ (เรียกรวมๆว่า Triplet) อีกหน้าที่หนึ่งของ Edges ก็คือ การ Send Messages ไปหาSource/Destination vertex ก็ได้ ใน triplet 2) **Aggregate Messages**: ทำการ Merge/combine Messages ที่เข้ามาเป็นคู่ๆ แล้ว return ออกมาเป็น single message ผลลัพธ์จะเป็น RDD of pairs (vertexID & Aggregate Messages)หรือนับจาก incoming edgesก็ได้

- เช่น ใช้ aggregateMessages สำหรับคำนวณ no. of followers

```
val followers = socialGraph.aggregateMessages[Int]
(edgeContext => edgeContext.sendToDst(1), (x,y) => (x + y))
```

คล้ายกับ code นับ incoming ... val followers = socialGraph.inDegrees

#### 5. Graph-parallel computation เช่น Pregel

ตอนทำ aggregateMessages คือทำรอบเดียวแล้วจบ แต่ถ้าเราอยากให้ algorithm ทำซ้ำหลายรอบและทำงานพร้อมกันหลายๆอันขนานกันไป จะได้ process เร็วๆ ก็ต้องใช้ Pregel เข้ามาช่วย

**Pregel**: เป็น framework (ทำเป็น Steps) ในการทำงานแบบขนาน ที่ถูก develop by Google เรียกว่า Google's Pregel Algorithm ซึ่ง based on Bulk Synchronous Parallel (BSC) algorithm

- ประกอบด้วย a sequence of iterations ที่เรียกว่า SUPERSTEPS

- Vertex จะ activated หรือทำงาน ก็ต่อเมื่อได้รับ message จาก vertex ก่อนหน้า โดยจะทำงานแบบ parallel กันไป แต่ถ้าไม่ได้รับ message ก็จะอยู่เฉยๆ ไม่ทำงานต่อแล้ว

- SuperSteps จะทำงานซ้ำ (iteration) วนไปเรื่อยๆ จนกว่าจะไม่มี vertex ไหนได้รับ sms อีกต่อไป จึงจะหยุดการทำงาน

- Pregel เหมาะกับการทำงาน Large-scale graph เพราะทำงานแบบขนาน ทำให้ทำได้เร็วกว่า

- Pregel ประกอบด้วย 2 sets of arguments

- The first set of arguments**: 1) Initial messages ที่แต่ละ vertex จะได้รับใน first superstep 2) Max iterations: กำหนดให้ทำซ้ำกี่รอบ 3) Direction: จะให้ส่งไปไหน (out/in)

- The second set of arguments**: 1) update vertex function คือการคำนวณค่า vertex property ใหม่แล้ว update 2) send function คือการคำนวณ optional messages สำหรับส่งไป vertex ที่อยู่ใกล้ๆถัดไป 3) aggregate function คือการ merges/ combines messages ที่ได้รับมา ซึ่งจะเรียงขั้นตอนใหม่ดังนี้ 1) Combine all incoming sms >> 2) Update vertex property>> 3) Send sms out

- Update program ประกอบด้วย 3 arguments ดังนี้ 1) VertexID, 2) Current properties of the vertex, 3) An aggregation of all messages

```
val firstMessage = 0.0
val iterations = 20
val edgeDirection = EdgeDirection.out
```

```
val influenceGraph =
inputGraph.pregel(firstMessage, iterations,
edgeDirection)(updateVertex, sendMsg,
aggregateMsgs)
```

```
val influenceGraph =
inputGraph.pregel(firstMessage, iterations,
edgeDirection)
((a,b,c) => // updateVertex
triplet => // sendMsg
(x,y) => // aggregateMsgs
```

#### 6. Graph Algorithm

- aggregateMessages / Pregel เป็นเครื่องมือที่เราจะต้องมา custom graph algorithm เอง แต่ GraphX มี algorithm สำเร็จรูป built-in มาให้ด้วย ซึ่งจะทำให้เราประหยัดเวลามากขึ้นในการทำงาน ไม่ต้องมานั่งเขียนเอง ซึ่ง algorithm เหล่านี้จะถูกเรียกใช้ผ่าน Graph class ได้แก่

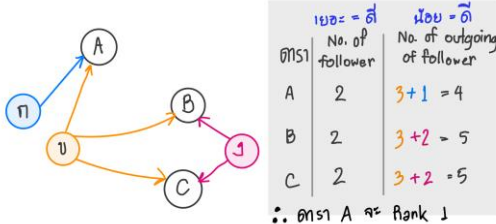
- 1) PageRank
- 2) Connected Components (undirected graph)
- 3) Strongly Connected Components (SCC) (directed graph)
- 4) TriangleCount



## 6. Graph Algorithm : graph algorithm สำเร็จรูป

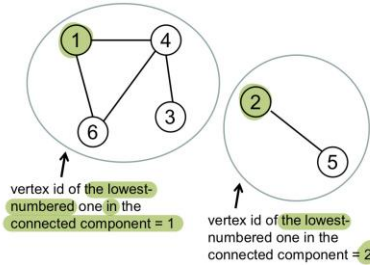
### 1. PageRank

- เป็นการจัดลำดับความสำคัญของ Web page / document ยิ่งถ้ามี link เข้าหาเยอะ Score ก็ จะสูง เช่น PageRank ใน Google เวลาเรา search คำต่างๆ จะขึ้น webpage ขึ้นมา โดยใช้วิธี PageRank
- ถ้าเอามาใช้ใน Social network เราจะระบุได้ว่าใครเป็น influencer โดย rank of all user จาก rank of the followers (คน follow เยอะยิ่งดี) และจำนวนคนที่แต่ละ followers กำลัง follow อยู่ หรือ no. of outgoing links of the follower (ถ้าคนที่มา follow เรา เราไม่ค่อยติดตามใคร ติดตามแค่เรา ก็ยิ่ง score สูง)



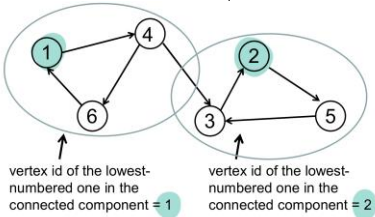
### 2. Connected Components (ใช้กับ undirected graph)

- Connected Components เดียวกันคือ ทุกๆ vertex จะมี edges เชื่อมหากันไปได้ทุก vertex
- Connected Components จะให้ค่า VertexID ที่น้อยที่สุดในบรรดา vertex ในวง



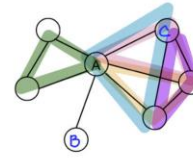
### 3. Strongly Connected Components (SCC) (ใช้กับ directed graph)

- คือ Subgraph ที่ทุกๆ vertices มี edge เชื่อมถึงกัน (Reachable) จากทุกๆ vertex ก็คือวนลูปไปครบทุกๆ vertex ได้
- จะให้ค่า VertexID ที่น้อยที่สุดในบรรดา vertex ในวง



### 4. TriangleCount

- ใช้สำหรับวิเคราะห์หา Strongly or isolated graph
- ใช้นับจำนวนสามเหลี่ยมที่เชื่อมกันเป็น loop ระหว่าง 3 vertex ใน graph
- ใช้เพื่อหาค่า Clustering Coefficient ซึ่งใช้วัดว่าใน social media network มีความเชื่อมโยงกัน เหนียวแน่นขนาดไหน (ยิ่งค่า CC สูง ยิ่งเหนียวแน่น)
- จากรูป TriangleCount for Vertex A = 4 เพราะมีสามเหลี่ยมทั้งหมด 4 รูปที่เชื่อมกับ vertex A



TriangleCount for vertex A = 4

$$\text{Clustering Coefficient} = \frac{\text{Triangle Count}}{\binom{d}{2}}$$

where d is the degree of \* of neighboring vertices of vertex

Ex

$$CC(A) = \frac{4}{\binom{6}{2}} = \frac{4}{\frac{6!}{2!4!}} = \frac{4}{15} = 0.26$$
$$CC(B) = 0$$
$$CC(C) = \frac{3}{\binom{4}{2}} = \frac{3}{\frac{4!}{2!2!}} = \frac{3}{6} = 0.5$$